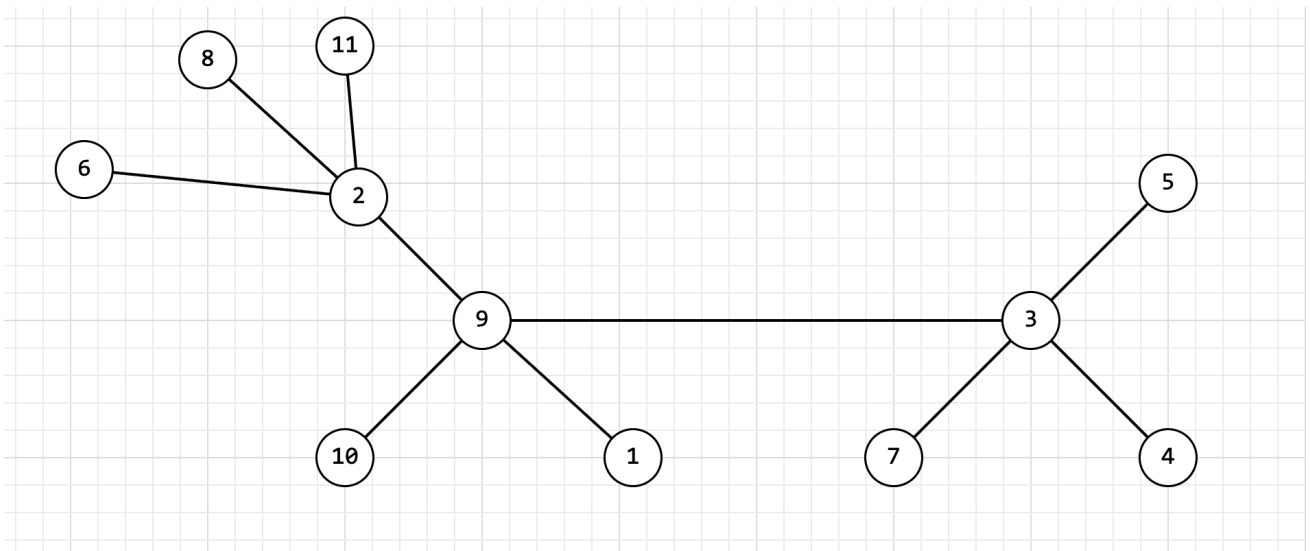


Д/з

1. Приведите пример графа с n вершинами и $n - 1$ ребром, который не является деревом.
2. Докажите, что любое дерево является двудольным графом.
3. Графовая последовательность дерева T имеет вид 5, 4, 3, 2, 1...1. Сколько в ней единиц?
4. Постройте все неизоморфные друг другу деревья на 5 и 6 вершинах.
5. Проиллюстрировать алгоритм на примере следующего графа.



6. Какова максимальная степень вершины в дереве, последовательность Прюфера которой имеет вид (1, 7, 2, 2, 2, 2).

История

- «Отец» теории графов Леонард Эйлер
- Задача о семи мостах Кенигсберга
- Письмо итальянскому математику и инженеру Маринони от 13 марта 1736 года. В этом письме Эйлер пишет о том, что он смог найти правило, пользуясь которым легко определить, можно ли пройти по всем мостам, не проходя дважды ни по одному из них (в случае семи мостов Кёнигсберга это невозможно).

Эйлеров граф / цикл

Мосты Кенигсберга – возникший в XIII веке город Кенигсберг формально состоял из трех независимых городских поселений, нескольких слобод и поселков. Они

располагались На берегах и островах реки Прегель, делившей город на четыре основные части: Альтштадт и Лёбенихт, Кнайпхоф, Ломзе, Фортштадт. Степень и вершины нечетные — невозможно пройти.

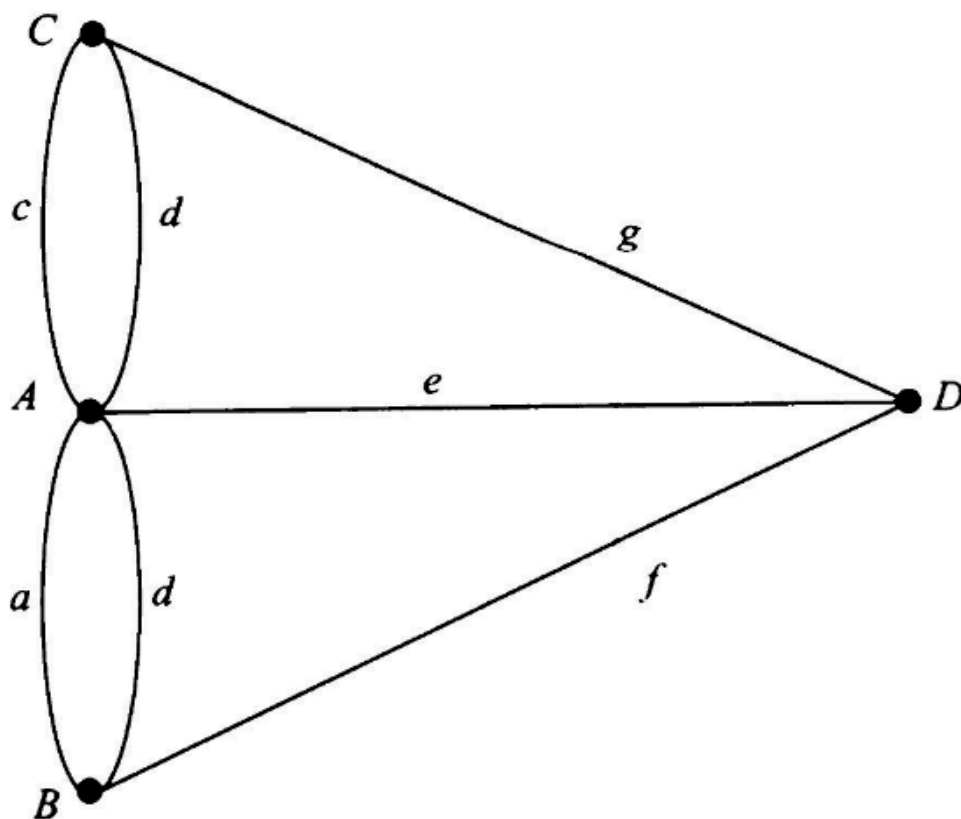


Рисунок 7.2. Модель задачи о мостах Кенигсберга

Эйлеровым путем в графе называется путь, который проходит по каждому ребру, причем ровно один раз.

Эйлеров цикл — замкнутый эйлеров путь.

Граф называется **эйлеровым**, если он содержит эйлеров цикл.

В связном графе существует эйлеров цикл тогда и только тогда, когда все его вершины имеют четную степень.

В связном графе существует эйлеров путь тогда и только тогда, когда количество вершин с нечетной степенью равно двум.

Если дана геометрическая фигура, как начертить ее на бумаге одним росчерком пера, не проводя дважды ни одну линию?

Прикладные задачи

Физик Густав Роберт Кирхгоф (1847 год):

- Теория графов для исследования электрических цепей
- Множество фундаментальных циклов в графе

Математик Артур Кэли (1857 год):

- Задачи органической химии
- Перечислить число предельных углеводородов с данным числом n атомов углерода
- На языке графов: найти число всех деревьев с заданным количеством вершин и степенями вершин 1 и 4

Гамильтон

Сэр Уильям Роуэн Гамильтон (1859 год)

- Головоломка «Вокруг света»
- Додекаэдр, каждому из 20 вершин приписано название города
- Обойти «вокруг света» по ребрам многогранника, посещая каждую вершину ровно один раз
- **Гамильтоновым путём** называется простой путь, проходящий через каждую вершину графа ровно один раз
- **Гамильтоновым циклом** называют замкнутый гамильтонов путь
- Граф называется **гамильтоновым**, если он содержит гамильтонов цикл.
- Достаточные условия гамильтоновости графа

Задача коммивояжера

Задача коммивояжера — задача, в которой коммивояжер должен посетить N городов, побывав в каждом из них ровно по одному разу и завершив путешествие в том городе, с которого он начал.

Туристические приложения

Задача — обойти как можно больше достопримечательностей на карте. Сервисы такие как Google Maps, Rome2Rio и туристические приложения используют TSP для построения оптимальных маршрутов.

Конвейер на заводе

Оптимизация маршрута перемещения деталей между станками или участками.

Например, робот на заводе должен пройти через несколько точек, эта задача сводится к TSP.

Маршруты курьеров

Логистика и доставка, то есть раздача заданий курьерам и планирование их маршрутов. Это прямая задача для курьерских служб и служб доставки, таких как DHL, FedEx и прочих. Например, такая система строит маршрут для конкретного курьера, чтобы он посетил все точки с минимальными затратами времени и топлива. Похожие задачи

решают службы доставки в электронных магазинах, торговых сетях и производственных предприятий. Они пытаются оптимизировать маршруты доставки заказов по городу, сбора товаров с разных складов.

Задача о домиках и колодцах

Три поссорившихся соседа имеют три общих колодца. Можно ли провести непересекающиеся дорожки от каждого дома к каждому колодцу?

Полный двудольный граф _{3,3}

Доказано, что данный граф непланарен, то есть задача о домиках и колодцах не имеет решения на плоскости

Критерий Понтрягина-Куратовского

Граф планарен тогда и только тогда, когда он не содержит подграфов, гомеоморфных $K_{3,3}$ и K_5

Задача о раскраске карт или гипотеза о четырех красках

1850-1852 г.г. Ф. Гатри постановка задачи: «сколько потребуется красок для раскраски любой плоской карты таким образом, чтобы никакие две смежные области не были окрашены в один и тот же цвет».

Гипотеза: достаточно четырех красок. [Ссылка](#)

Огастес Де Морган (23 октября 1852 года), профессор математики Университетского колледжа в Лондоне, написал письмо своему другу сэру Уильяму Роуэну Гамильтону, выдающемуся ирландскому математику и физику. В нем впервые была упомянута гипотеза о четырех красках.

Сегодня один мой студент попросил объяснить ему утверждение, о котором я ранее не знал и в верности которого до сих пор не уверен. Он говорит, что если фигура каким-либо образом разделена на части и эти части окрашены в разные цвета так, что фигуры, имеющие общую границу, окрашены в разный цвет - может потребоваться четыре цвета, но не более - в его случае требуется четыре цвета.

Вопрос: нельзя ли придумать пример, в котором возникнет необходимость в пяти или более цветов? Что вы скажете? И было ли это утверждение замечено ранее, если оно верно? Мой студент говорит, что догадался об этом раскрашивая карту Англии...

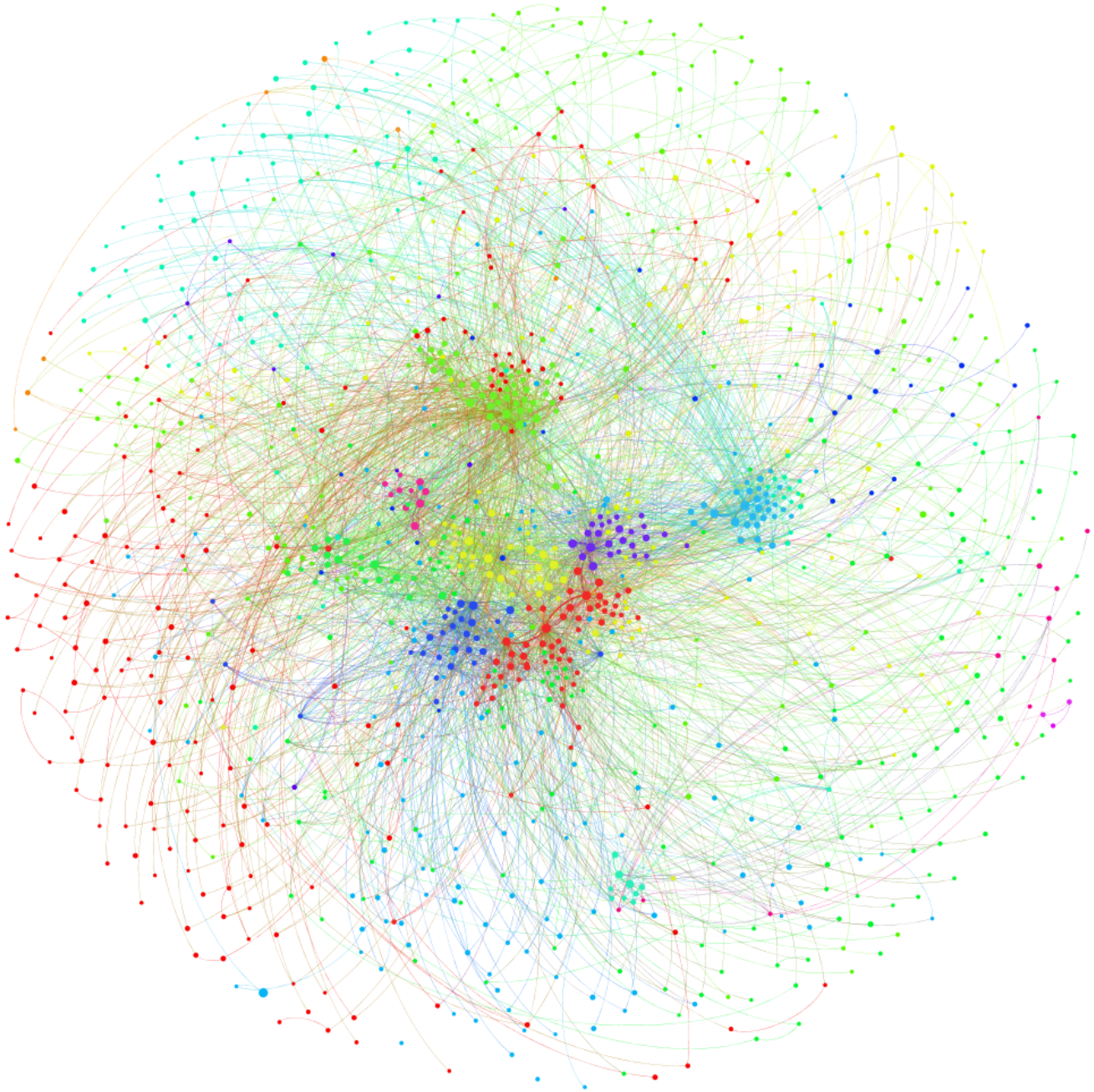
- 1879 г. британский математик Альфред Брей Кемпе дал ошибочное доказательство
- 1890 г. лектор Дурхэмского университета Перси Джон Хивуд опровергнул доказательство Кемпе и показал, что гипотеза верна для пяти красок
- 1925 г. Филипп Франклин, оставив в стороне общую проблему четырех красок, сумел доказать, что для раскрашивания любой карты, содержащей не более 25 областей, требуются только четыре краски.

- 1940 г. Винн распространил доказательство на карты с 35 областями
- 1970 г. Оре и Стемпл увеличили число областей до 39
- 1975 г. известный популяризатор науки и многолетний ведущий раздела «Математические игры» журнала «Scientific American» Мартин Гарднер опубликовал карту, для раскрашивания которой якобы требовались пять красок.
- 1976 г. два математика из Иллинойского университета, Вольфганг Хакен и Кеннет Appel, предложили новый метод, перевернувший традиционные представления о математическом доказательстве.
- Хакену и Appelю удалось свести проблему четырех красок «всего лишь» к 1482 конфигурациям, служащим теми строительными блоками, из которых можно составить любую карту.
- В июне 1976 года, затратив 1200 часов машинного времени, Хакен и Appel заявили во всеуслышание, что им удалось проанализировать все 1482 карты и для раскрашивания ни одной из них не требуется более четырех красок.

Теория графов в игре престолов

Граф социальной активности

Вершинам соответствуют персонажи. Размер вершины зависит от сказанных данным персонажем слов. Ребра - диалоги персонажей. Толщина ребра зависит от количества состоявшихся разговоров.



Граф неориентированный. Вершин (персонажей): 1105. Рёбер (диалогов): 3505
5 книг, почти 2 миллиона слов. Для сбора данных было привлечено машинное обучение
(точность определения принадлежности разговора составляет около 75%).

Алгоритмы на графе

Подсчет степеней вершин - самые многосвязные персонажи.

Диаметр графа (расстояние между наиболее удаленными вершинами графа) равен 8.

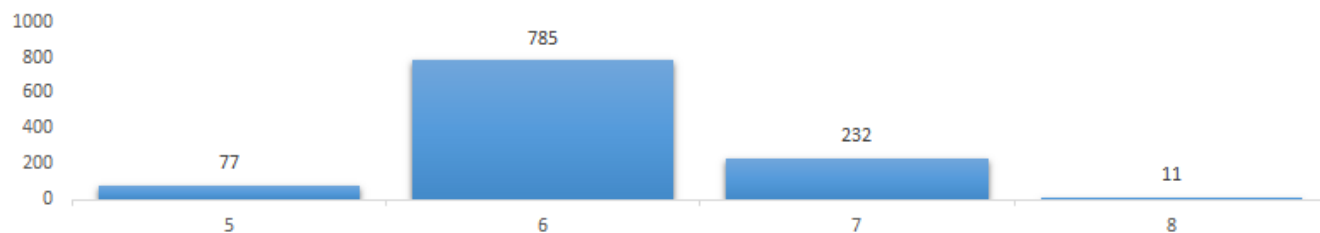
Поиск путей: 77 персонажей находятся «в центре» графа, иными словами они могут
связаться с любым другим персонажем не более чем через 4х других.

Нахождение максимальных клик (9 персонажей). Остовное дерево - главные связи.

Мосты и точки сочленения.

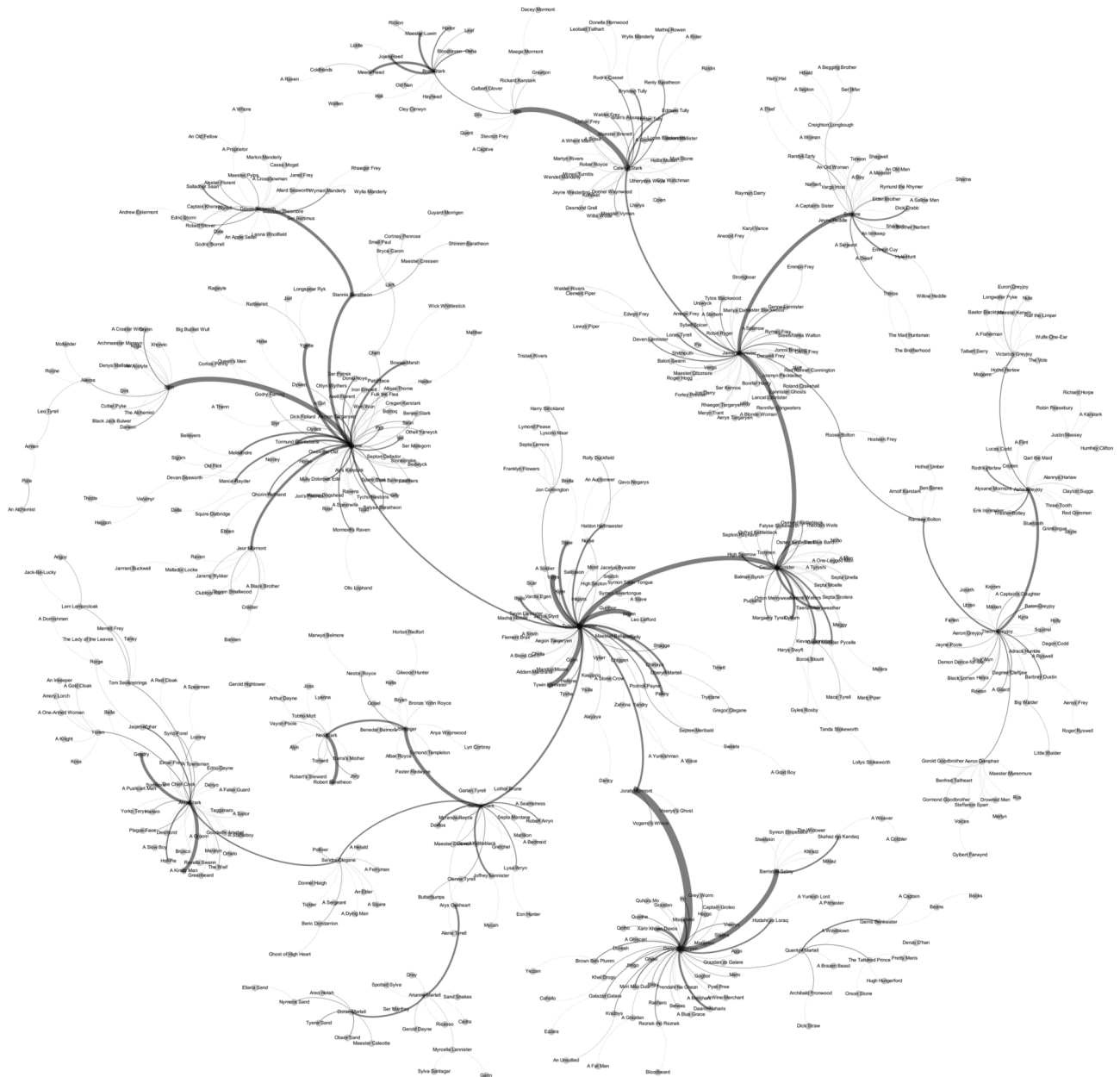
Минимальный разрез (сколько героев требуется «убить» чтобы полностью исключить контакт двух самых популярных (по полученной статистике) персонажей).

Распределение длин путей



Остовное дерево (алгоритм Краскала)

Главные герои и их связи



Исследование графа – модели социальной сети

Степени вершин.

Центральность (различные подходы).

Расстояние между вершинами, диаметр графа.

Плотность.

Кластеризация и выявление сообществ.

Механизмы формирования (модели сетей).

Диффузия в сетях (эпидемии, убеждения, инновации) - процесс, при котором какой-то сигнал постепенно распространяется по всей сети.

Предсказание связи.

Социальные сети

Отношения между людьми:

- Сеть дружбы
- Сеть соавторства ученых
- Браки между кланами
- Бизнес-отношения
- Совместное появление киноактеров в фильмах
- Телефонные звонки, email'ы, сеть icq-контактов

Информационные сети

Информационные сети – это сети отношений между информационными объектами.

1. Цитирования в научных статьях.

Цитирования в научных статьях (citations) образуют граф, в котором вершины — это научные статьи, а ориентированные ребра означают, что одна статья ссылается на другую.

2. Цитирование в патентах.

3. Совместное употребление слов в текстах.

Технологические сети

Технологические сети показывают «физические» связи в нашем трехмерном мире.

- Интернет как сеть компьютеров
- Национальные электросети
- Телефонные линии, почтовые службы доставки
- Поезда, самолеты, автобусы

Биологические сети

Сети внутри между животными растениями, людьми:

- Сеть нейронов в мозге
- Реакции между протеинами
- Кровеносные сосуды
- Реки, озера, океаны
- Граф "жертва-хищник"

«Мир тесен»

(серия экспериментов американского социолога Стэнли Мигрэма – поиск и анализ средней длины пути между объектами социальных сетей)

«Двух случайно взятых людей отделяет друг от друга цепочка из не более чем 6 звеньев (или рукопожатий)»

«Kevin Bacon's game»: каждый американец подсчитывал, сколько рукопожатий отделяют его от Кевина Бэкона. Например, Кевина Бэкона отделяют от Чарли Чаплина всего три звена: сам Бэкон играл с Лоуренсом Фишберном, который в свою очередь играл с Марлоном Брандо, который снимался с Чарли Чаплином.

Число Эрдеша

Поль Эрдеш имеет индекс ноль.

Его соавторы имеют индекс 1.

Соавторы соавторов имеют индекс 2, и т.д.

Гипотеза: не бывает определенного значения индекса Эрдеша больше 15.

Факт: у 98% математиков индекс Эрдеша не более 7.

Например: Paul Erdos - Richard K. Guy - Юрий Матиясевич - Юрий Лифшиц

Алгоритм PageRank

PageRank был представлен и опубликован Сергеем Брином (Sergey Brin) и Ларри Пейджем (Larry Page) на седьмой международной конференции World Wide Web (WWW7) в апреле 1998 года.

Это алгоритм ранжирования с использованием гиперссылок в Интернете. На основе алгоритма, они построили поисковую систему Google

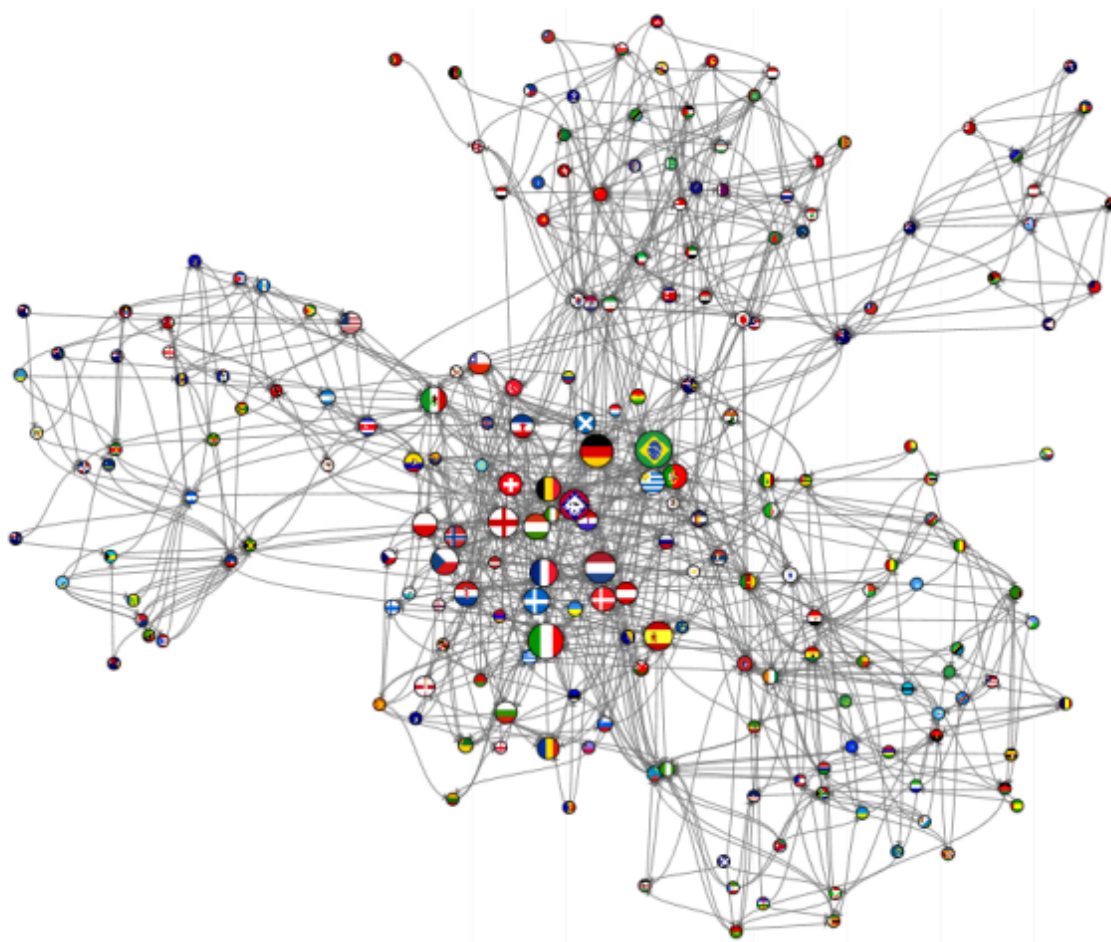
PageRank – это числовая величина, характеризующая «важность» веб-страницы. Чем больше ссылок на страницу, тем она становится «важнее». Таким образом, **PageRank** – это метод вычисления веса страницы путём подсчёта важности ссылок на неё.

Алгоритм PageRank и футбольные команды

Вершины графа – команды, ребра - матчи.

Вес ребра зависит от результата матча.

Определение наиболее успешных команд с помощью PageRank.



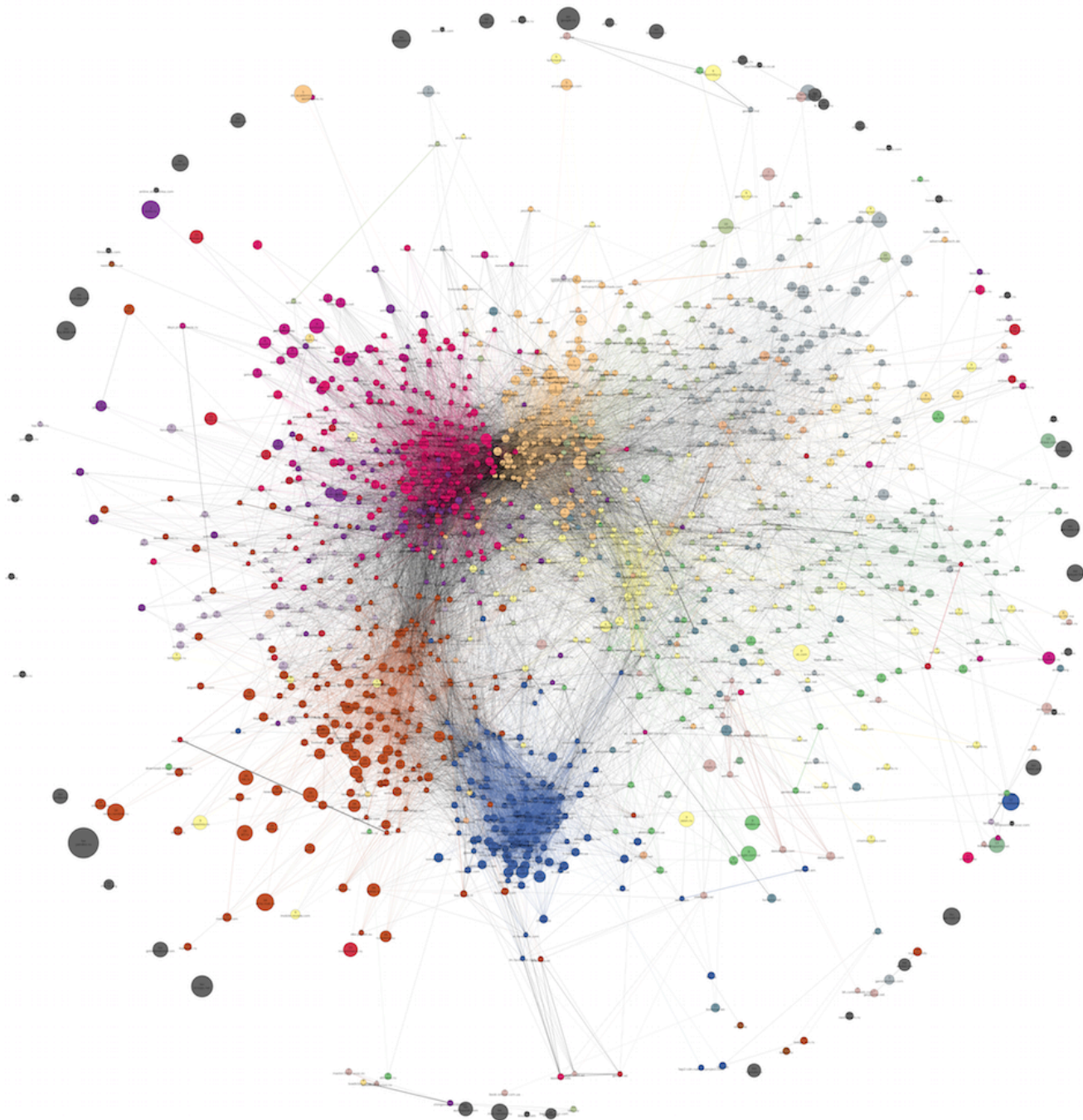
Поиск сообществ в графах

Задачи кластеризации

Выделение сообществ (кластеров) разных объектов: пользователей, сайтов, продуктовых страниц интернет-магазинов:

1. Выделение сегментов пользователей для проведения рекламных кампаний.
2. Использование кластеров в качестве предикторов (прогностический параметр, средство прогнозирования) в персональных рекомендациях
3. Снижение размерности в любой задаче машинного обучения.
4. Сличение товарных URL между различными интернет-магазинами с целью выявления среди них групп, соответствующих одному и тому же товару.
5. Компактная визуализация – человеку будет проще воспринимать структуру данных.

[Поиск сообществ в неориентированных графах \(Граф доменов рунета – 1285\)](#)



Вершины – домены. **Рёбра** – аффинити между доменами. **Аффинити между доменами** – это выборочная оценка того, насколько события «посещение юзером домена А» и

Дан социальный граф, число тестовых вершин 262588, число ребер в графе 9437519, число вершин в графе 1862220. Данный граф получен из реального путем удаления ребер.

Задача: для заданных тестовой выборкой пользователей предсказать до 10 других пользователей, которых им бы стоило зафолловить.

Математическая модель интернета

Веб-граф (А. Райгородский)

Вершинами этого графа будут сайты, и между двумя вершинами А, В мы проведем столько ребер, направленных от А к В, сколько есть ссылок с сайта А на сайт В, и столько ребер, направленных от В к А, сколько есть ссылок с сайта В на сайт А.

Свойства веб-графа

Диаметр веб-графа равен 6 (закон шести рукопожатий)

Веб-граф является достаточно разреженным: если вершин у веб-графа n , то ребер у него не более n с некоторым постоянным ≥ 1

Характерное распределение степеней вершин (аналогичное транспортным, биологическим и т.д. сетям)

Таким образом, несмотря на кажущуюся хаотичность в процессе образования интернета, есть весьма жесткие статистические ограничения, которым он годами подчиняется.

Почему это так? Что стоит за всеми свойствами интернета? Каковы законы, управляющие формированием сети?

Теория графов и реконструкция генома

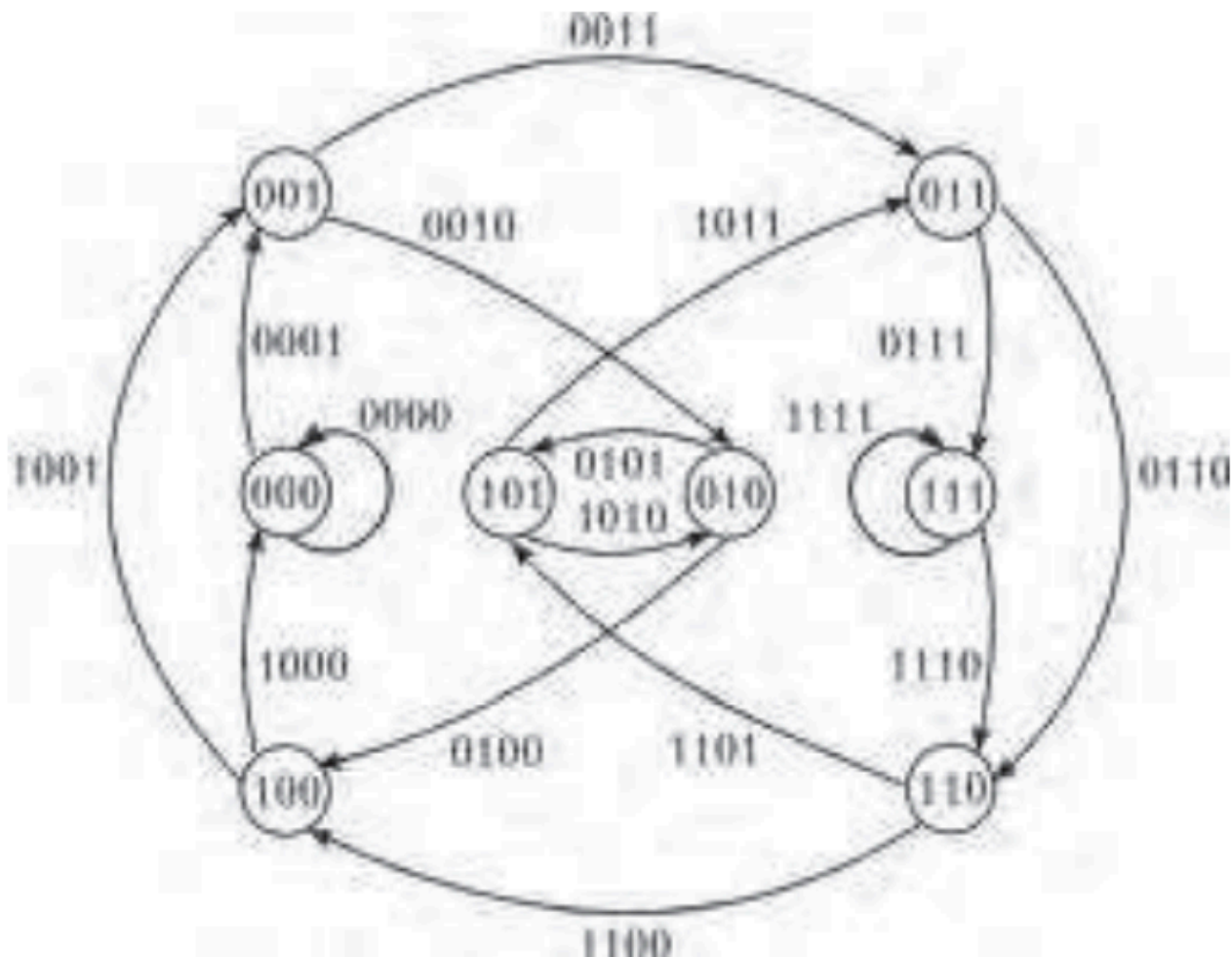
Задача ассемблирования (сборки) геномов из так называемых ридов – относительно коротких (порядка 100 символов) строк над 4-буквенным алфавитом (A,C,G,T), получаемых в результате секвенирования (разделения) генома (а точнее большого количества одинаковых геномов).

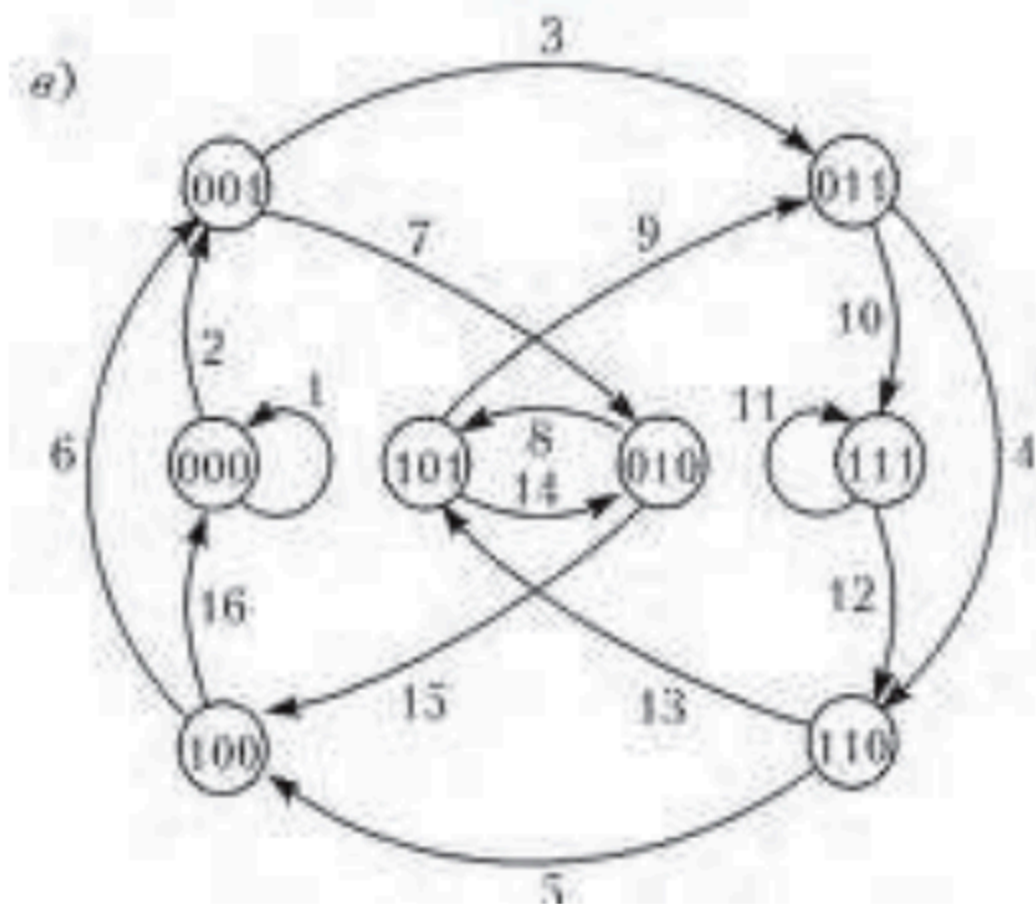
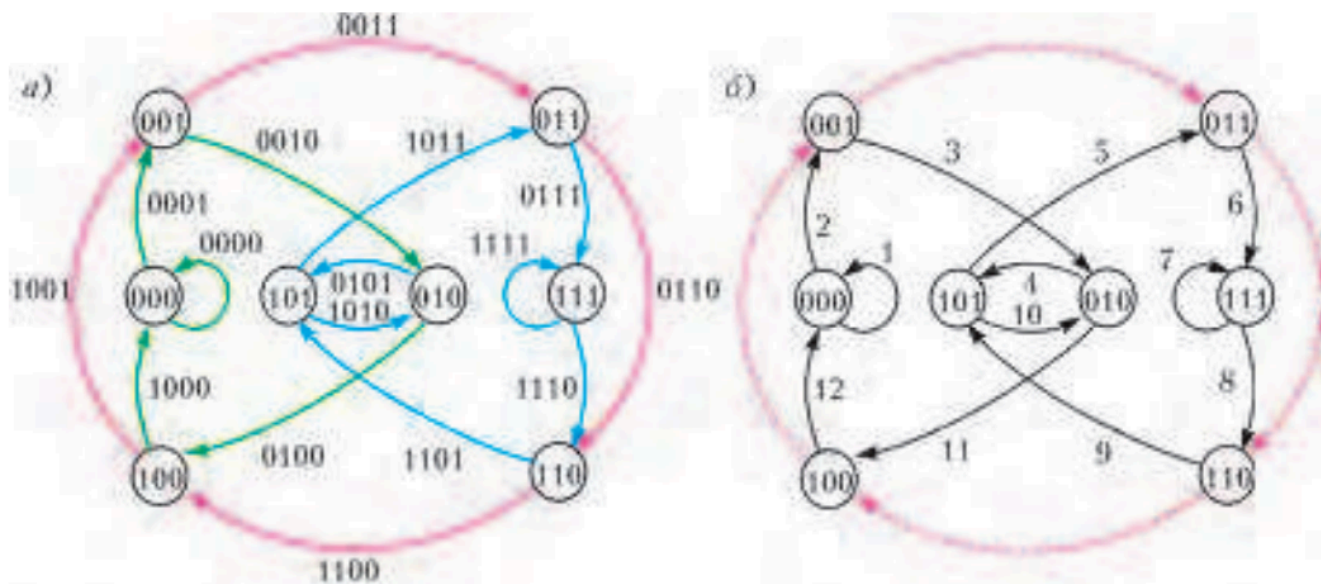
Граф де Брейна:

Алфавит из n букв, число k .

Вершины - все возможные слова длины $k-1$.

Ребро из вершины x в вершину y строится тогда, когда существует k -буквенное слово, для которого x является префиксом, y – суффиксом. Все графы де Брейна имеют эйлеров цикл.





Поиск эйлерова цикла в графе. Сначала разбиваем граф на циклы (а). Затем по очереди объединяем эти циклы, применяя описанный выше алгоритм (б). В итоге получится единственный эйлеров цикл (в); цифры на ребрах обозначают порядок обхода. В нашем случае цикл дает строку 0000110010111101.

[Баланс в настольном геймдизайне](#)

"Письма призрака" – детективная настольная игра с тайными ролями на дедукцию, блеф и ассоциативное мышление. Основная механика "Писем призрака" строится на ассоциациях между картами с изображениями различных предметов (картами улики). Задача балансировки ассоциаций состояла примерно в следующем:

1. минимизировать количество сильных "однозначных" ассоциаций. Каждая карта в идеале должна примерно с равной силой ассоциироваться с несколькими другими.
2. избежать "обособленных групп ассоциаций". Не должно быть групп карт, которые ассоциируются только друг с другом и не ассоциируются ни с какими другими картами.
3. добиться максимальной равномерности и хаотичности распределения ассоциаций по всему множеству карт.

[illegible]

Построение графа (Gephi)

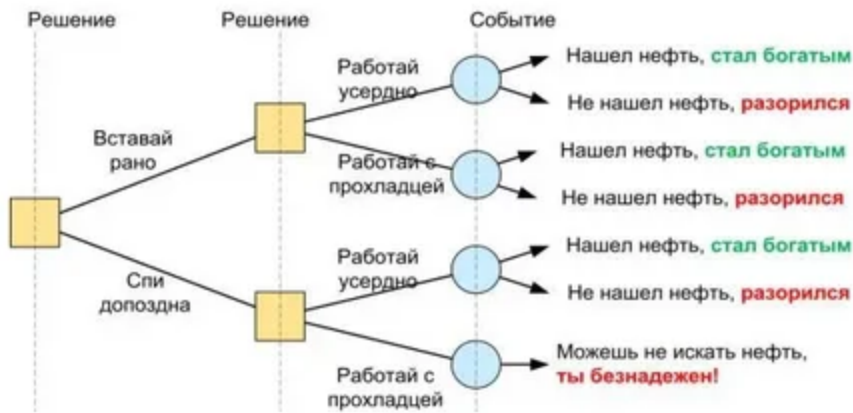
Наглядное размещение вершин (алгоритмы укладки).

Толщина ребра - сила связи (отображаются только связи для 2 и 3).

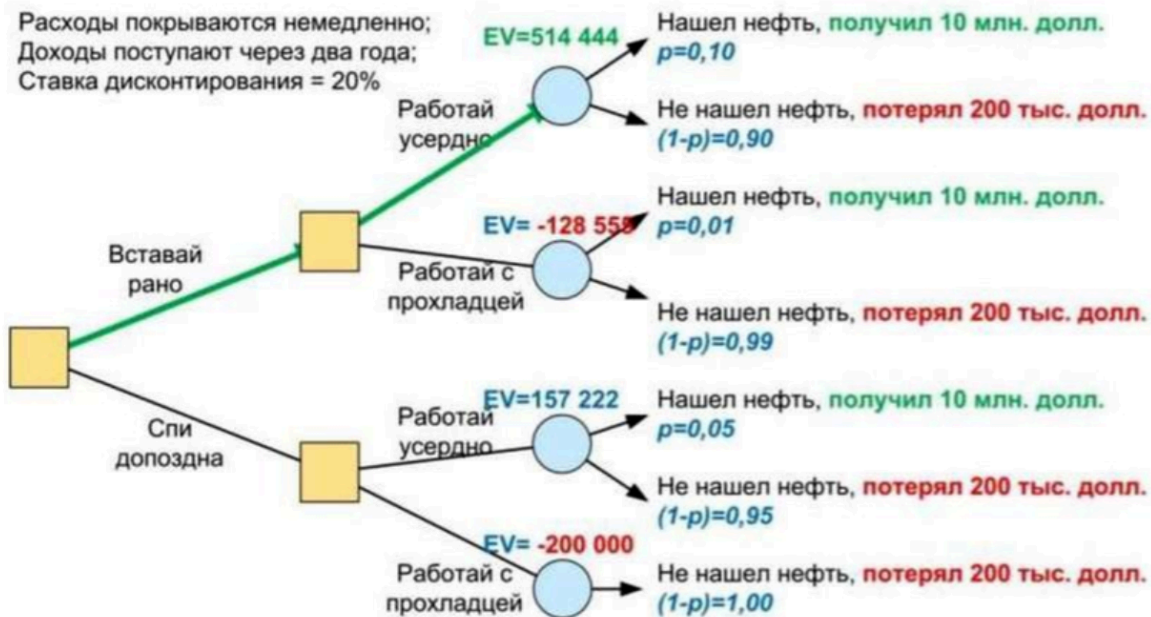
Размер вершины зависит от количества связей от нее.

Раскраска вершин по кластерам (граф должен быть связным, количество вершин в кластерах приблизительно одинаково и т.д.).

Последовательность (дерево) принимаемых решений



Дерево решений с ожидаемыми значениями (EV) приведенных результатов (долл.)



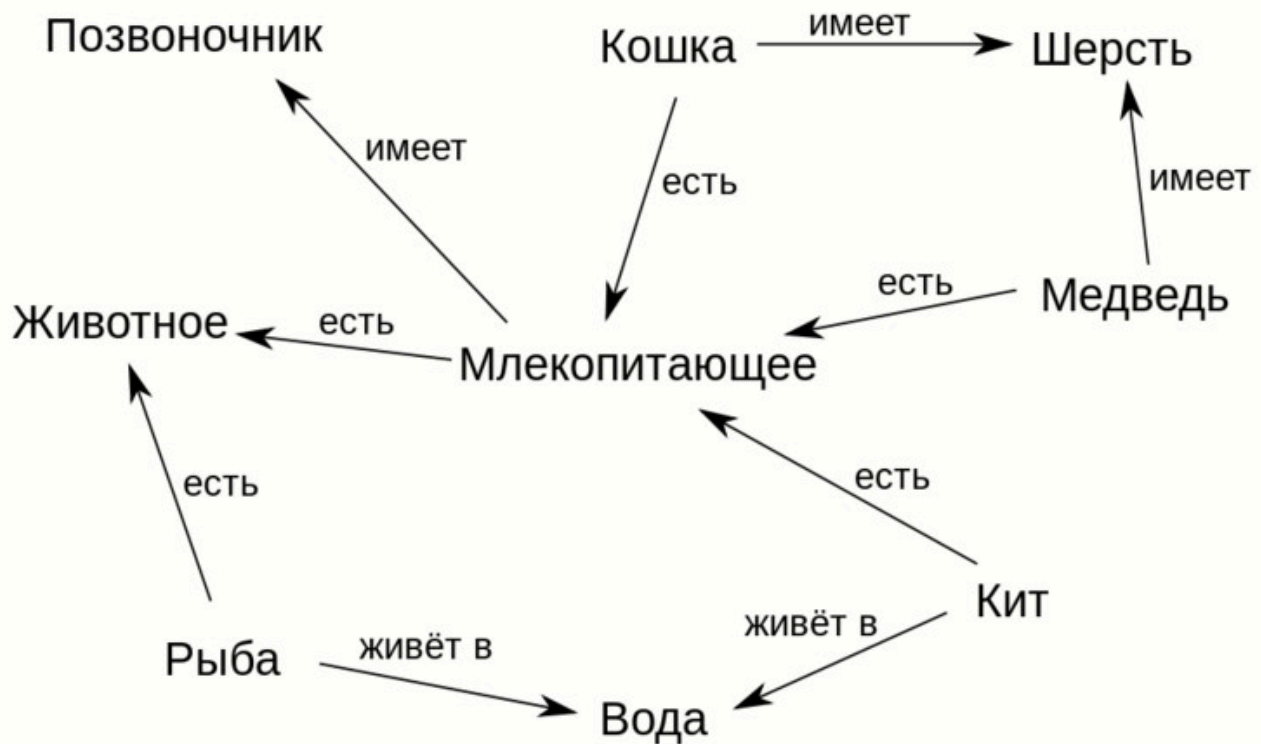
Графы знаний

Представление знаний (англ. knowledge representation) направление в исследованиях искусственного интеллекта, посвященное представлению информации о мире в форме, которую было бы возможно использовать в компьютерных системах для решения сложных прикладных задач.

Пример: диагностирование заболеваний или ведение диалога на естественном языке.

- 1960 г. – семантические сети
- 1980 г. – Гронингенский университет и университет Твенте начали работу над совместным проектом, названным "Графы знаний", базируясь на устройстве семантических сетей с рёбрами, ограниченными наперёд заданным количеством отношений - для упрощения алгебры на графах
- 2012 г. – Google представили свою версию графа знаний

Пример семантической сети



Knowledge Graph, или граф знаний, – семантическая сеть, в которой хранится информация о разных сущностях взаимосвязях между ними. Сущностью или «узлом» графа может быть что угодно: человек, предмет, дата, концепция и любой другой материальный объект или абстрактное понятие. Предикаты или «ребра» отражают взаимосвязи и отношения между разными сущностями в графе. Например, Альберт Эйнштейн и город Ульм в Германии – две самостоятельные сущности, а тот факт, что Эйнштейн родился в Ульме, – будет уже предикатом.

Knowledge Graph помогают связать большое количество данных из разных источников в одну общую коллекцию знаний. Они могут быть общими, то есть хранить информацию о разных видах данных, и специализированными – фокусироваться на какой-то одной предметной области.

Например, Wikidata – один из самых крупных графов знаний – хранит разностороннюю информацию. BioPortal крупнейший специализированный граф с более чем 140 млрд фактов о биотехнологиях и медицине.

Число открытых Knowledge Graph в интернете растёт, все вместе они образуют облако семантически связанных друг с другом данных – Linked Open Data Cloud.

[Граф знаний Google](#)

Используется Google и его службами для улучшения результатов поисковой системы с помощью информации, собранной из различных источников.

Граф знаний предоставляет структурированную и подробную информацию о теме в дополнение к списку ссылок на другие сайты. Цель состоит в том, что пользователи смогут использовать эту информацию для решения своих запросов без необходимости перехода на другие сайты и сбора информации самостоятельно. Например, при запросе «Леонардо Да Винчи» в правом углу будет отображена информация о художнике, краткие биографические данные, а также его картины, другие художники того же времени и т.д. При переходе на картину, для нее будет открываться своя расширенная страница, если информация о ней имеется в базе.

[Интерактивная карта открытых графов знаний различной тематики](#)

Графы и машинное обучение

<https://academy.yandex.ru/handbook/ml/article/grafovye-nejronnye-seti>

<https://deepschool-pro.notion.site/1665b8940c27431f95a801da2577c072>

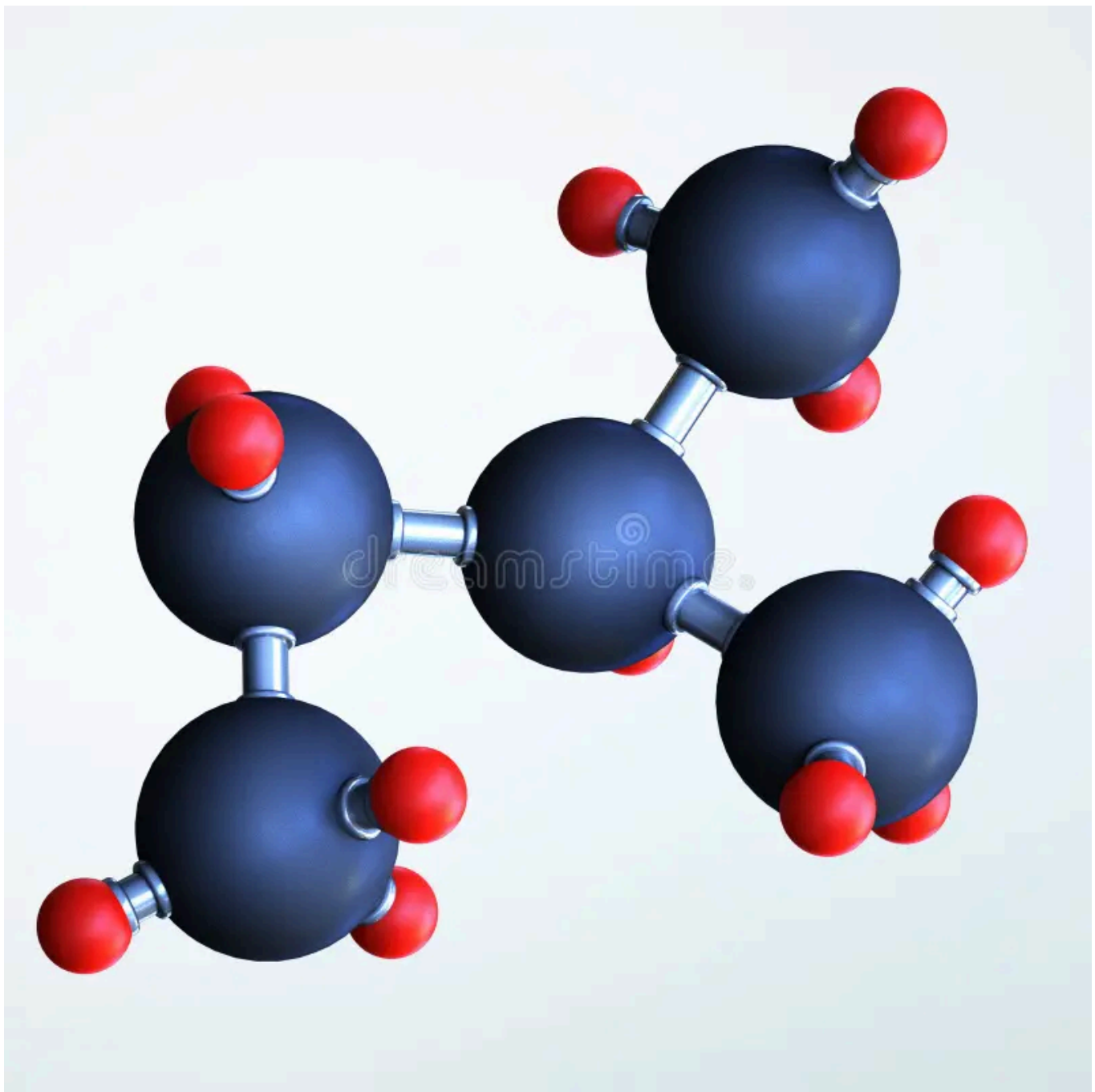
Довольно много данных, имеющих графовую структуру:

- описания дорожных и компьютерных сетей,
- социальные графы и графы цитирований,
- молекулярные графы, а также графы знаний, описывающих взаимосвязи между сущностями, событиями и абстрактными категориями.

Молекулярный граф

Молекулярные графы – это пример графов с, как правило, средним количеством вершин и ребер; вершины и связи в молекулярных графах имеют признаковое описание (типы атомов и ковалентных связей, а также информацию о зарядах и т.п.), но при этом также являются гомогенными графами.

(вершины - атомы, а ребра – ковалентные связи)



Графовые данные довольно разнообразны. Они могут отличаться между собой в следующих моментах:

- По размеру, т.е. количеству вершин и/или ребер.
- По наличию признаков описаний вершин и рёбер. В зависимости от решаемой задачи, графы могут содержать информацию только в вершинах, только в ребрах, либо же и там и там.
- Кроме того, графы могут быть **гомо-и гетерогенными** - в зависимости от того, имеют ли вершины и ребра графа одну природу либо же нет.

К классу гетерогенных графов относятся, например, **графы знаний**. Вершины (сущности) и связи (ребра) такого графа могут иметь различную природу: скажем,

вершинами могут быть сотрудники и подразделения компании, а рёбра могут отвечать отношениям «Х работает в подразделении У», «Х и Z коллеги» и так далее.

Социальный граф

Социальные графы содержат огромное количество вершин и ребер, часто измеряющееся в тысячах, содержат информацию в вершинах и очень редко в ребрах, а также являются гомогенными, так как все вершины имеют один тип.

Задачи на уровне всего графа

Прогнозирование свойства целого графа. Например, для молекулы, представленной в виде графа, мы можем предсказать ее потенциальное взаимодействие с рецептором, связанным с заболеванием или какие-то химический свойства.

Для изображений – задачи классификации изображения, для текста, например, определение тональности.

Задачи на уровне вершин

Связаны с прогнозированием свойств или роли каждой вершины в графе.

Например, предсказать интересы нового пользователя по интересам его друзей.

Классическим примером задачи прогнозирования на уровне вершин является "Каратэ-клуб Зака". Набор данных представляет собой единственный социальный сетевой граф, состоящий из лиц, которые присягнули на верность одному из двух каратэ-клубов. По легенде, конфликт между мистером Хай (инструктором) и Джоном А (администратором) вызвал раскол в каратэ-клубе. Вершины - это каратисты, а ребра - взаимодействия между этими каратистами за пределами каратэ. Задачей прогнозирования является классификация по тому, станет ли данный участник преданным мистеру Хай или Джону Х после конфликта. В данном случае расстояние от вершины до инструктора или администратора сильно коррелирует с этой меткой.

Задачи прогнозирования на уровне вершин идентичны сегментации изображений, где мы пытаемся присвоить класс каждому пикселю на изображении.

В случае текста, аналогичная задача – прогнозирование частей речи для каждого слова в предложении (например, существительное, глагол, наречие и т.д.).

Задачи на уровне ребер

Например, предсказания пропущенных связей в графе. В больших графах часто некоторые связи отсутствуют. Например, в социальном графе пользователь может добавить не всех знакомых в друзья. А в графе знаний могут быть проставлены только простые взаимосвязи, а высокоуровневые могут быть пропущены.

Оптимизация «классических» алгоритмов

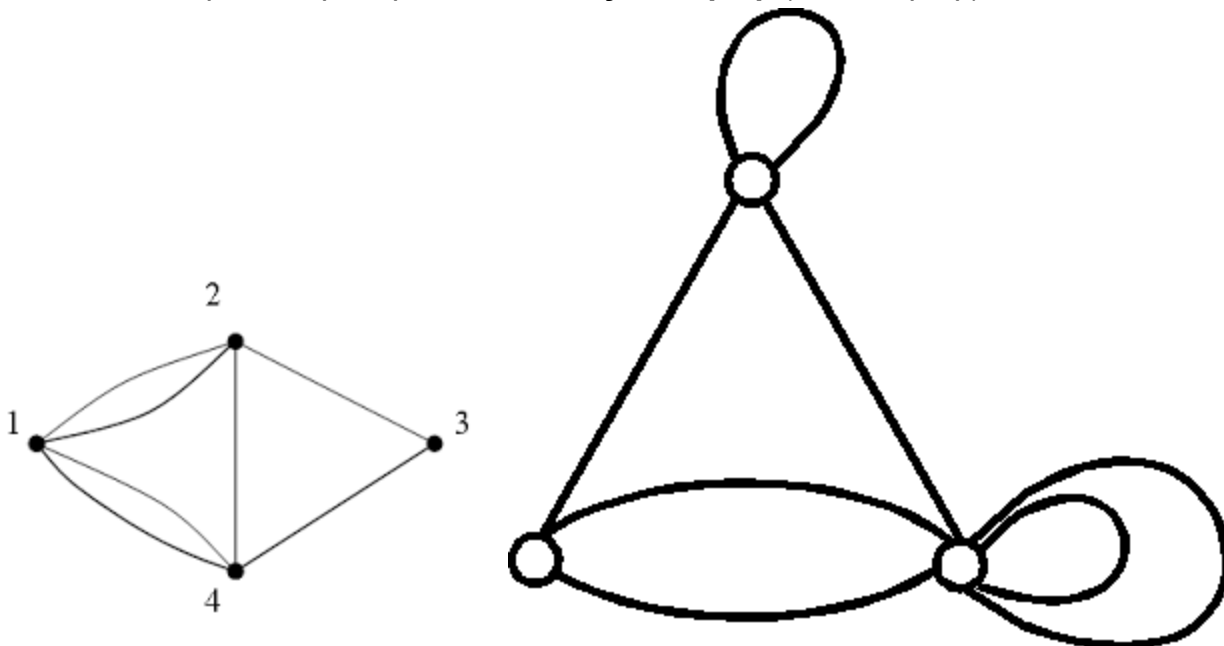
Группа исследователей из университетов Синьхуа, Стенфорда и Института Макса Планка представили [детерминированный алгоритм](#) для решения задачи SSSP в ориентированных графах с неотрицательными вещественными весами, который работает за время, пропорциональное числу ребер, умноженному на логарифмический множитель, который растет медленнее, чем обычный логарифм.

Проблема поиска кратчайшего пути от одной вершины до всех остальных (SSSP) — одна из фундаментальных в теории графов, и её история тянется с 50-х годов прошлого века. Классический алгоритм Дейкстры, в связке с продвинутыми структурами данных, решает эту задачу за время, которое примерно пропорционально сумме числа ребер и произведения числа вершин на логарифм от их же числа.

Именно этот множитель – число вершин, умноженное на логарифм, долгое время считался теоретическим минимумом, так как в своей основе алгоритм Дейкстры побочно сортирует вершины по расстоянию от источника. Этот предел известен как «барьер сортировки» и казался непреодолимым.

Основные определения

Граф (простой) — пара $G = (V, E)$, где V — конечное множество вершин, E — множество пар вершин. Если E — множество неупорядоченных пар вершин, то граф называется **неориентированным** (элементы множества называются ребра), иначе — **ориентированным** (элементы множества называются дуги). Возможность кратных ребер и петель — **мультиграф** (псевдограф).



Если есть ребро (x, y) , то вершины x, y — концевые, при этом говорят, что ребро *инцидентно* своим концевым вершинам. В неориентированном графе вершина y смежна с вершиной x , если существует ребро (x, y) . Отношение смежности для неориентированного графа симметрично: если x смежна с y , то y смежна с x . В ориентированном графе вершина y смежна с вершиной x , если существует дуга (x, y) .

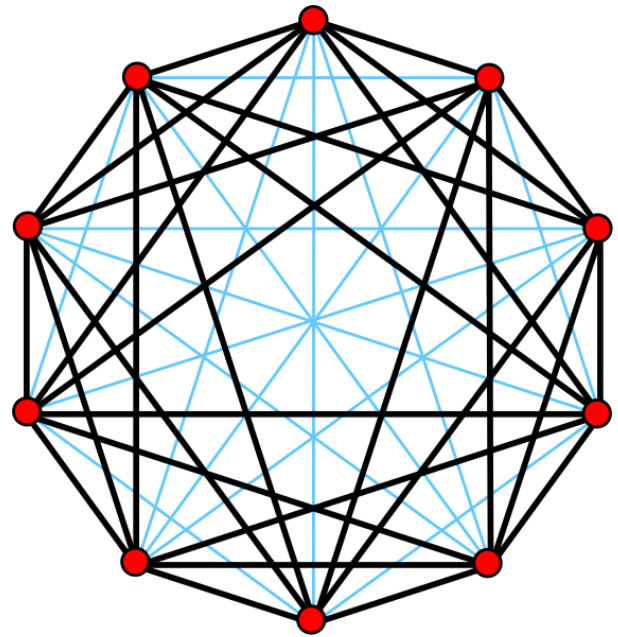
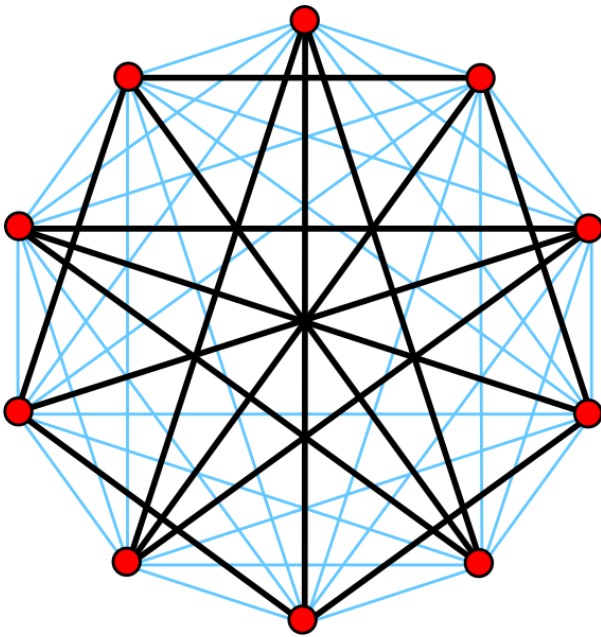
В неориентированном графе степень (x) или валентностью вершины x называется количество ребер, инцидентных x . Если вершина имеет степень 0, то она называется **изолированной**. В орграфе различают входящую ($in(x)$) и исходящую ($out(x)$) степень вершины.

Граф называется **полным**, если каждая его вершина смежна со всеми остальными.

Полный граф с n вершинами обозначается K_n .

Дополнением графа называется граф, у которого множество вершин совпадает со множеством вершин графа, а множество ребер дополняет множество ребер графа до полного графа.

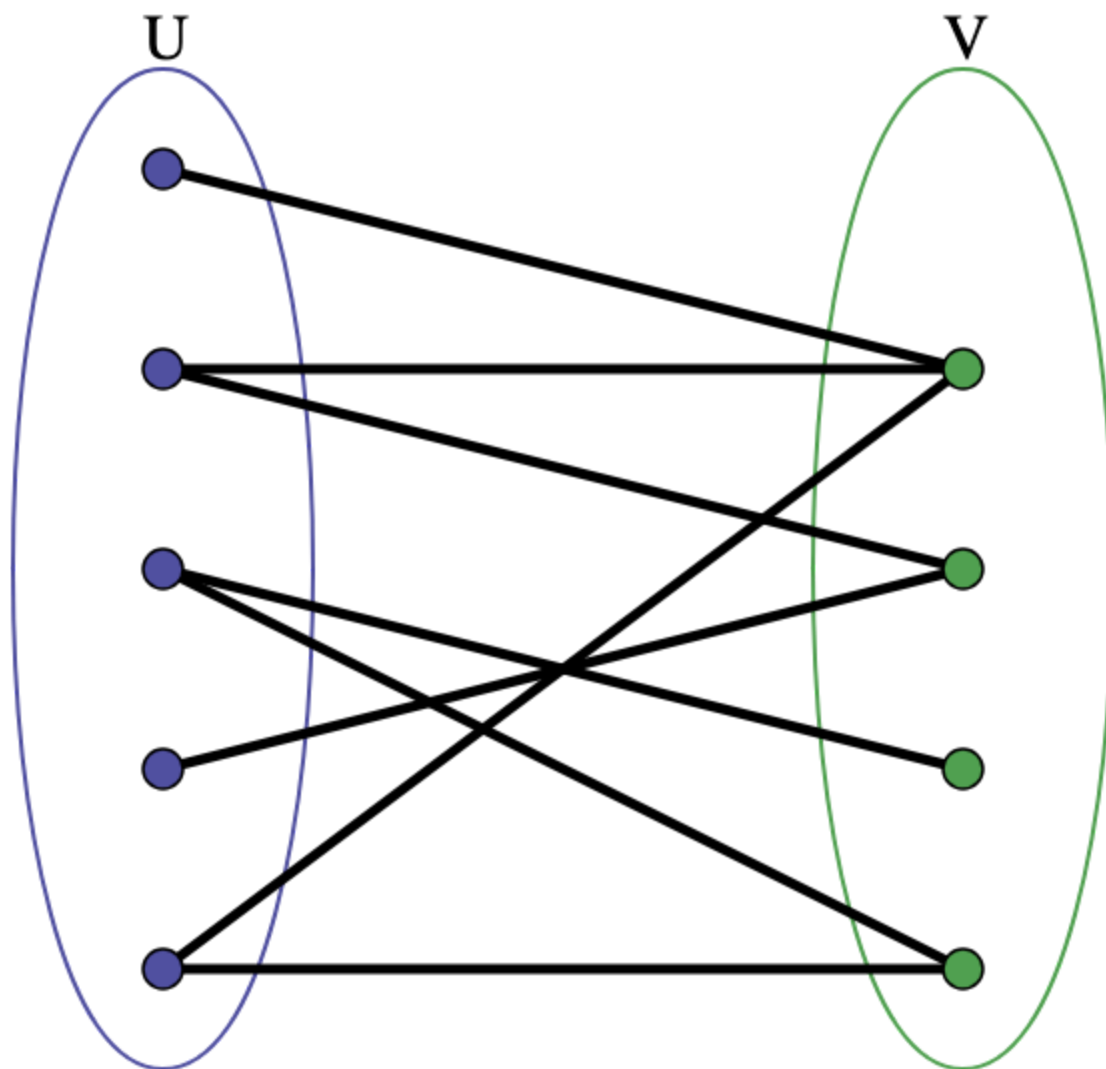
Дополнением полного графа является пустой граф, то есть граф, все вершины которого изолированные.



Граф $G = (V, E)$ называется **двудольным**, если множество его вершин можно разбить на два непересекающихся подмножества таким образом, что любое ребро будет иметь своими концами вершины из разных подмножеств.

$$V = V_1 \cup V_2, V_1 \cap V_2 = \emptyset$$

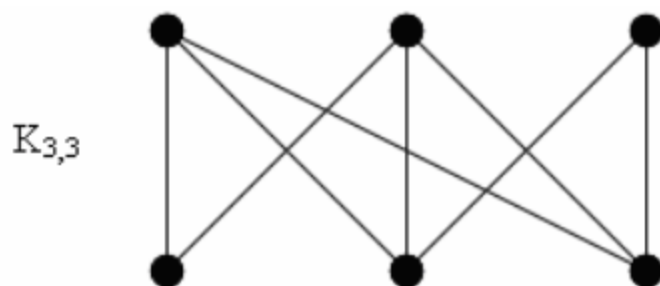
$(x, y) \in E \Rightarrow x \in V_1, y \in V_2 \text{ или } x \in V_2, y \in V_1$



Двудольный граф называется **полным**, если каждая вершина из U_1 соединена ребром с каждой вершиной из U_2 .

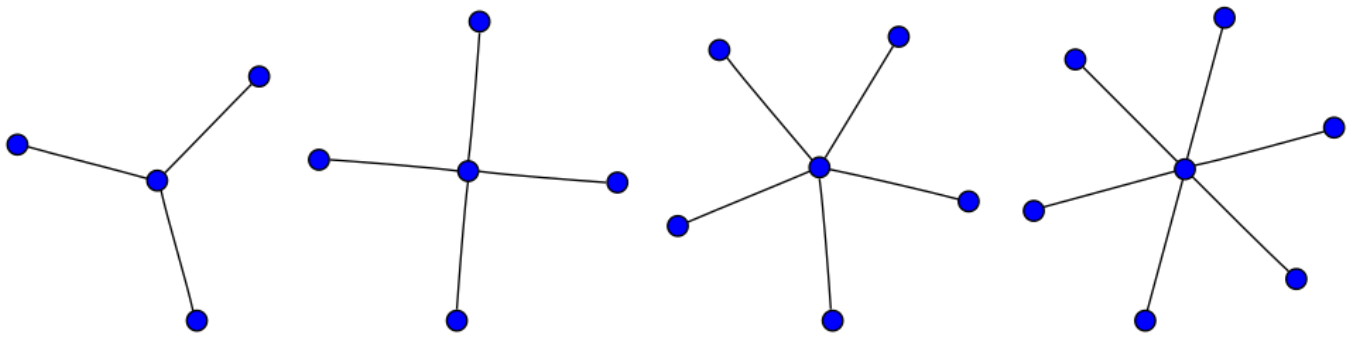
Полный двудольный граф обозначается $K_{n,m}$

где $|U_1| = n$, $|U_2| = m$

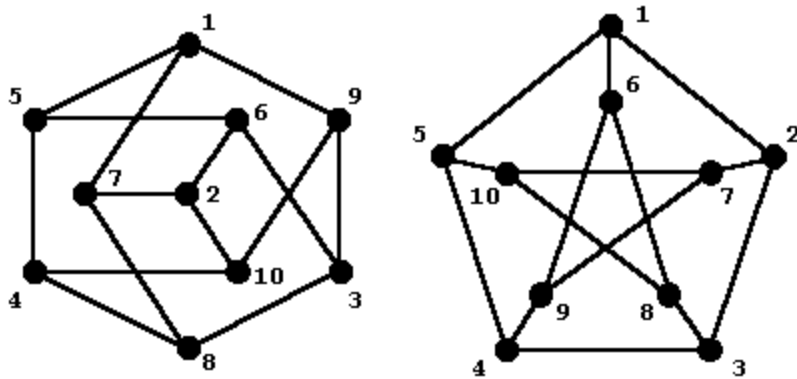


$K_{3,3}$

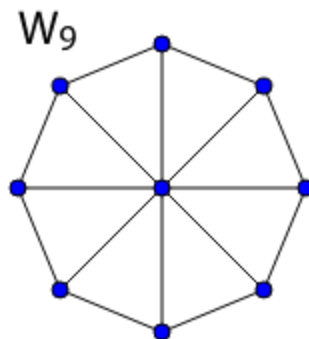
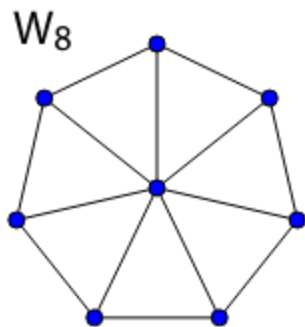
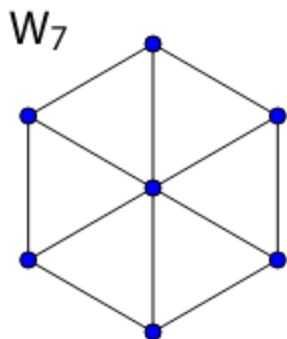
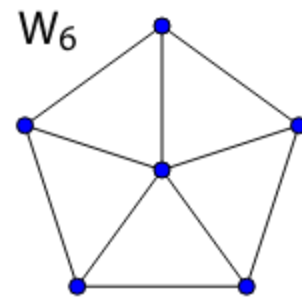
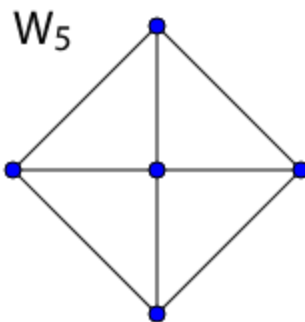
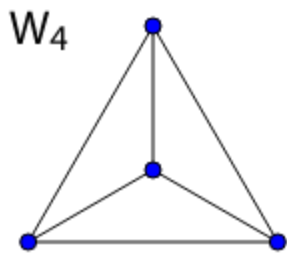
Граф $K_{n,1}$ называется звездой.



Два графа $(1, 1)$ и $(2, 2)$ называются **изоморфными**, если существует биекция (взаимоднозначное соответствие) $\phi: V_1 \rightarrow V_2$, такая что $(u, v) \in E_1$ тогда и только тогда, когда $(\phi(u), \phi(v)) \in E_2$

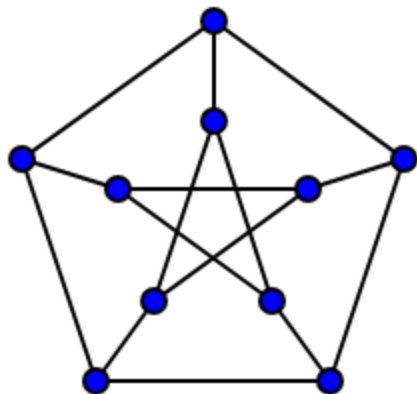


Путь n , цикл n , n .



Регулярный граф степени k (-регулярный) – это граф, все вершины которого имеют степень k . Если $k = 3$, такой граф называется **кубическим**.

Граф Петерсена



Теорема (Эйлер, 1737 год)

1. В графе сумма степеней вершин равна удвоенному числу ребер (лемма о рукопожатиях).

2. Число вершин нечетной степени четно.

$$\sum d(v) = 2E$$

$$\sum_{d(v) \text{ нечет}} d(v) = 2E - \sum_{d(v) \text{ чет}} d(v)$$

$\sum_{d(v) \text{ нечет}} d(v) = 2E - \sum_{d(v) \text{ чет}} d(v)$, справа получается четное число, а слева сумма нечетного числа слагаемых, она будет четной если их количество будет четное число.

3. Найдется по крайней мере две вершины, имеющие одинаковую степень.

n - количество вершин, сколько может быть в таком графе степеней вершин? В графе с n вершинами может быть от 0 до $n - 1$ степеней вершин, однако не может быть одновременно степень 0 и степень $n - 1$, это значит что максимальное число вершин для $n - 1$ степень.

Замечание

Для полного графа есть n и каждая вершина соединена с $n - 1$ вершиной значит количество всех ребер $\frac{n(n-1)}{2}$.

Полный двудольный граф – размерность первой доли n_1 , размерность второй доли n_2 , значит количество всех ребер $n_1 \cdot n_2$. В двудольном графе сумма степеней вершин одной доли равна сумме степеней вершин второй доли.

Аналог леммы о рукопожатиях для орграфа

В орграфе сумма исходящих степеней всех вершин равна сумме входящих степеней всех вершин и равна числу ребер.

Игры и графы

Перед началом игры имеется группа, содержащая некоторое нечетное число предметов, например 25. Играют двое, ходят по очереди. Сделать ход - это значит отделить от общей группы 1, 2, 3 или 4 предмета. В процессе игры общая группа уменьшается, а каждый из играющих накапливает предметы. Победителем считается тот, кто сумеет к концу игры накопить четное число предметов. Беспроеигрышная стратегия состоит в том, чтобы вступить в игру вторым и действовать в дальнейшем так, как это предписывается графом, изображенным на рисунке 52.

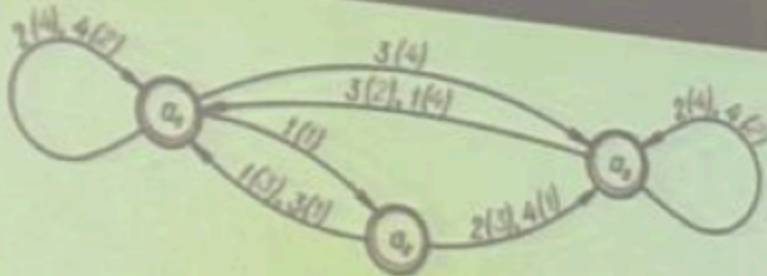


Рис. 52

Игра начинается в состоянии a_1 . Первым в ней выступает игрок В.
 Пусть игрок В делает ход: «беру 3 предмета». В ответ игрок А ходит: беру 4 предмета» и для себя замечает, что игра перешла из состояния a_1 в состояние a_2 .
 Ситуация после первого хода:

Таблица 20

Номер хода	ход В	ход А	В какое состояние переходит игра
1	3	4	a_2

Второй ход игрока В: «беру 4 предмета». Ответный ход А: «беру 2 предмета» (игра остается в состоянии a_2 — так подсказывает петля).

Ситуация после второго хода:

Таблица 21

Номер хода	ход В	ход А	В какое состояние переходит игра
1	3	4	a_2
2	4	2	a_2

Пути в графах

Маршрутом в графе из вершины x , в вершину x , называется последовательность

$x_0, 1, x_1, 2, x_2, \dots, x_{i-1}, x$ вершин x_i и ребер i , где $i = (x_{i-1}, x_i)$

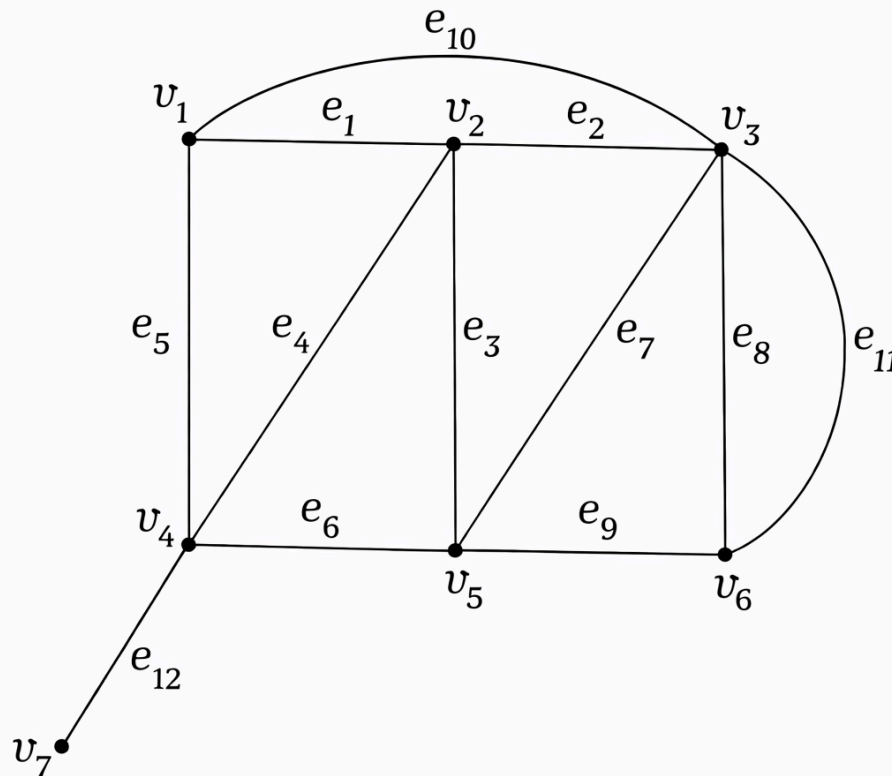
И вершины, и ребра в такой последовательности могут повторяться.

Количество ребер в маршруте называется **длиной маршрута**.

В случае простого графа любой маршрут полностью определяется последовательностью $x_0, x_1, x_2, \dots, x_{i-1}, x$ в которой любые два соседних элемента являются смежными вершинами, то есть (x_{i-1}, x_i)

Начальная вершина – x_0 , конечная вершина – x , внутренние вершины – $x_1, x_2, \dots, x_{i-1}$.

Если все ребра в маршруте различны, то такой маршрут называется **путем** из вершины x_0 , в вершину x . Если также и все вершины в данном пути различны, то такой путь называется **простым**. **Замкнутым путем** называется путь, в котором начальная и конечная вершина совпадают. **Циклом** называют замкнутый простой путь.



Маршрут – $1 \xrightarrow{1} 2 \xrightarrow{3} 5 \rightarrow 4 \xrightarrow{4} 2 \xrightarrow{2} 3 \rightarrow$

Путь – $1 \xrightarrow{5} 4 \rightarrow 5 \xrightarrow{7} 3 \xrightarrow{11}$

Простой путь – $1 \xrightarrow{10} 3 \rightarrow$

Замкнутый путь – $1 \xrightarrow{1} 2 \xrightarrow{3} 5 \rightarrow 4 \xrightarrow{5} 1$

Цикл (незамкнутый путь) – $1 \rightarrow 3 \rightarrow \rightarrow 5 \rightarrow 4 \rightarrow 1$

Если две вершины графа соединены хотя бы одним путем, то такие вершины называются **связанными**.

Связанность задает отношение эквивалентности на множестве вершин графа. Это отношение делит граф на классы эквивалентности, которые называются **компонентами связности графа**.

Если в графе существует лишь одна компонента связности, то есть, любые две вершины соединены хотя бы одним путем, то такой граф называется **связным**.

Расстоянием (x, y) между двумя связанными вершинами x, y называется **длина наименьшего пути между ними** (понятно, что такой путь обязательно является простым).

В том случае, когда вершины не являются связанными, полагают, что $(x, y) = \infty$

Диаметром графа $diam(G)$ называется максимальное расстояние между его вершинами. В случае несвязного графа полагают, что $diam(G) = \infty$.

Эксцентриситетом вершины называется максимальное расстояние от нее до любой вершины графа.

Радиусом графа $r(G)$ называется минимальный из эксцентриситетов вершин графа, а вершины, на которых этот минимум достигается, называются **центральными вершинами**.

Множество всех центральных вершин называется **центром графа**.

Подграфы

Подграфом графа G называется граф H , для которого выполнены следующие условия:

1. $V(H) \subseteq V(G)$
2. $E(H) \subseteq E(G)$;

Можно сказать, что любой подграф H графа G – это граф, полученный из исходного, с помощью последовательного выполнения двух операций:

1. Удаление ребра

Множество вершин не меняется, из множества ребер удаляется элемент e .

Полученный граф обозначается $G - e$. Можно сразу удалить несколько ребер, принадлежащих некоторому подмножеству M множества $E(G)$. Полученный граф обозначается $G - M$.

2. Удаление вершины x

Если эта вершина является изолированной, то кроме ее удаления ничего больше делать не нужно. Если же этой вершине инцидентны какие-то ребра, то будем также вместе с вершиной x удалять и их. Полученный в результате удаления вершины x граф обозначается обычно $G - x$.

В более общем случае мы с помощью операции удаления вершин из подмножества M получаем подграф $G - M$, в котором по сравнению с исходным графом удалены все вершины подмножества M вместе со всеми ребрами, инцидентными этим вершинам.

Частные случаи

1. Если мы при получении из графа G подграфа H используем лишь операцию удаления ребер, то мы получаем подграф, множество вершин которого совпадает с множеством вершин исходного графа.

Такой подграф называется **остовным подграфом (spanning subgraph)** графа .

В частности, любой простой граф , построенный на n вершинах, является **остовным подграфом полного графа K_n** .

2. Когда мы в графе удаляем одну или несколько вершин. Подграфом H графа , **индуцированным подмножеством вершин** , называется граф, полученный из удалением всех вершин, не принадлежащих множеству , вместе со всеми инцидентными этим вершинам ребрами.

Подклассы остовных подграфов

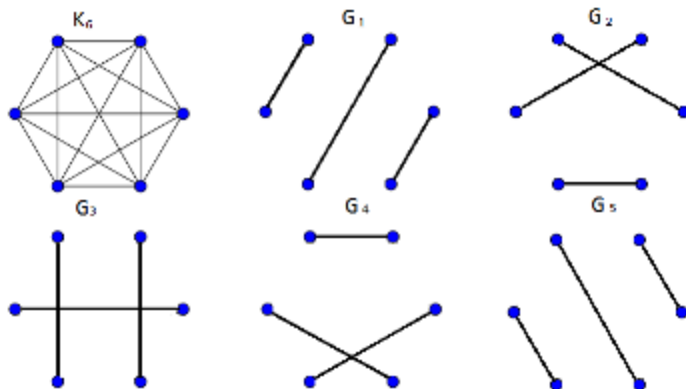
Остовный 1 -регулярный подграф для произвольного натурального 1 называется **1 -фактором** графа .

Остовный 1 -регулярный подграф носит название **1 -фактор** графа , а набор ребер в таком подграфе называется **совершенным паросочетанием** в исходном графе .

В случае мультиграфа очень часто рассматривают простой граф, получающийся из исходного графа удалением всех петель и заменой мультиребер на простые ребра.

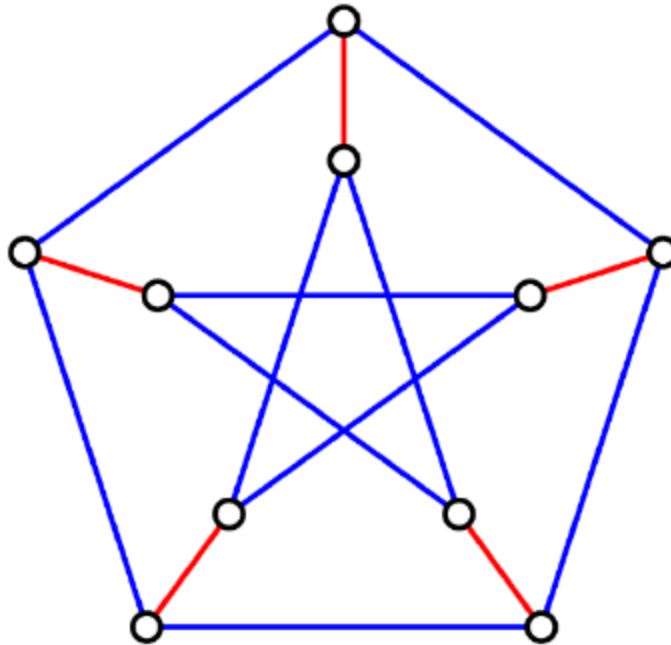
Такой граф, очевидно, является **остовным подграфом исходного графа**.

1 -факторизация графа



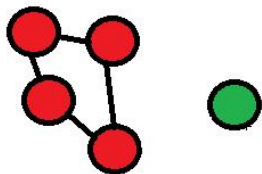
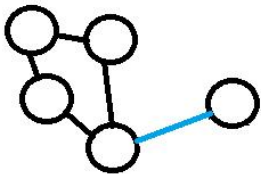
Паросочетанием в произвольном графе называется любой набор ребер, не имеющих общих концевых вершин. Паросочетание M называется **совершенным**, если оно покрывает все вершины графа.

1-фактор красный
2-фактор - синий



Ребро в связном графе называется мостом, если получающийся после его удаления граф — e становится **несвязным**. В случае несвязного графа мостом называется ребро, после удаления которого количество компонент связности увеличивается на единицу.

Вершина x называется **точкой сочленения графа**, если после ее удаления количество компонент связности графа — x увеличивается по сравнению с количеством компонент связности исходного графа.



Утверждения

Утверждение. Вершина x в связном графе, построенном на $n \geq 3$ вершинах, есть точка сочленения графа тогда и только тогда, когда в графе существуют отличные от x вершины u и v , такие, что x содержится в каждом пути из u в v .

Доказательство:

1. Пусть x есть точка сочленения в связном графе. Предположим, что утверждение неверно: для любой пары вершин u, v , найдется путь, не проходящий через x . Тогда в

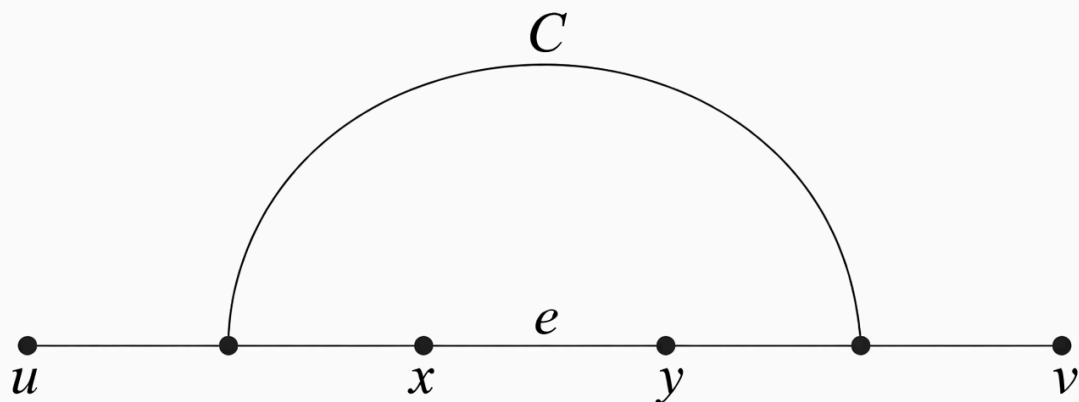
графе — x любая пара вершин остается связанной. Противоречие с определением точки сочленения.

- Предположим, что в G существует описанная в утверждении пара вершин u и v . Удаление вершины x в таком графе уничтожает любой путь из u в v . Вершины u и v становятся несвязанными. Теперь они будут принадлежать разным компонентам связности. То есть удаление x увеличивает количество компонент связности.

Утверждение. Ребро $e = (x, y)$ в простом связном графе G является мостом тогда и только тогда, когда e не принадлежит ни одному из циклов графа G .

Доказательство:

- Пусть e — мост. По определению удаление e приводит к появлению в графе $G - e$ хотя бы одной пары u, v не связанных между собой вершин. В исходном графе G эти вершины связаны между собой простым путем $P = (u, \dots, v)$, который с необходимостью проходит через $e = (x, y)$. В графе $G - e$ вершины u и x связаны между собой участком $(u, \dots, x)$ пути P , а вершины y и v участком $(y, \dots, v)$ пути P . Если бы ребро e принадлежало циклу C в графе G , то x и y оказались бы связанными в графе $G - e$ путем $C - e$. Но тогда связанными бы оказались и вершины u, v . Противоречие.
- Пусть $e = (x, y)$ не принадлежит ни одному из циклов графа G . Предположим, что e не является мостом, т. е. его удаление оставляет граф $G - e$ связным. Тогда x и y в графе $G - e$ связаны (путем P). Добавим к нему ребро e , получим цикл. Противоречие.



Сколько существует различных графов, построенных на n вершинах?

Заметим, что в случае мультиграфов мы наряду с количеством вершин должны также фиксировать и количество ребер. Действительно, любые две вершины мультиграфа можно соединить произвольным количеством ребер, поэтому количество мультиграфов, построенных на n вершинах, без ограничения на количество ребер может быть сколь угодно большим. Поэтому мы ограничимся задачей подсчета простых графов.

Количество n , простых неориентированных графов достаточно легко сосчитать, используя формальное определение простого графа как некоторого подмножества множества ⁽²⁾.

Действительно, количество всех двухэлементных подмножеств n -элементного множества вершин (то есть мощность множества ⁽²⁾) равно $|^{(2)}| = \binom{n}{2}$. Нас же интересует множество всех подмножеств ⁽²⁾. Количество элементов в этом множестве, как известно, равно

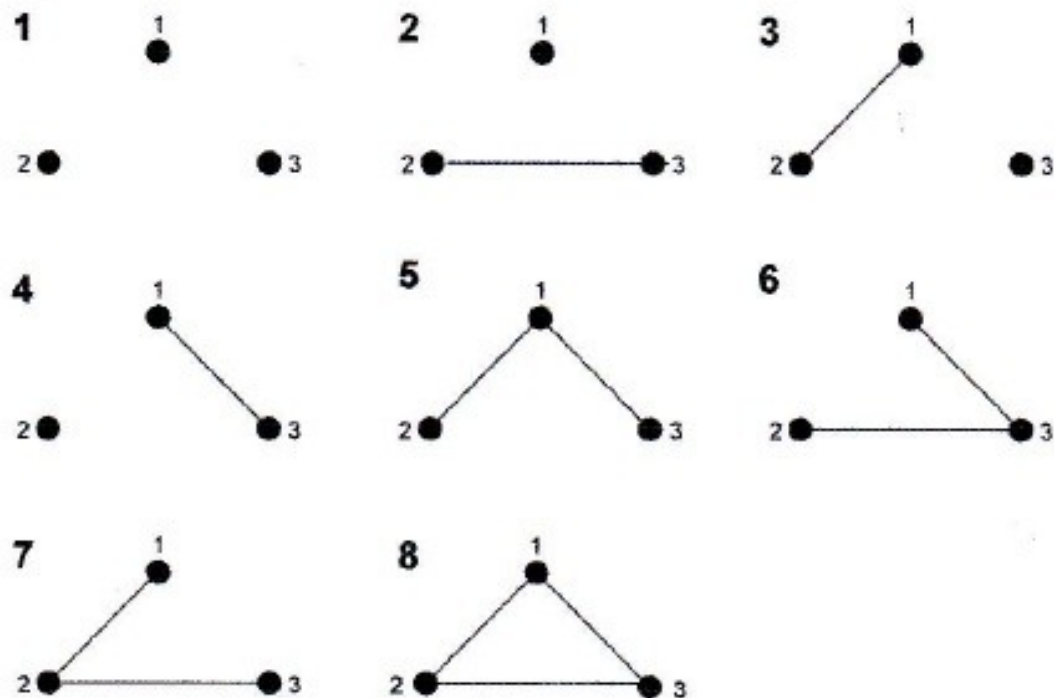
$$| | = 2^{|^{(2)}|} = 2^{\binom{n}{2}}$$

Следовательно, количество n , всех простых графов на n вершинах равняется $2^{\binom{n}{2}}$.

Для орграфов:

Всего существует $n(n-1)$ упорядоченных пар отличных друг от друга вершин. Как следствие, количество различных простых орграфов, построенных на n вершинах, равно $2^{n(n-1)}$.

Изоморфизм графов вводит отношение эквивалентности на множестве всех неориентированных простых графов на n вершинах. Отношение эквивалентности, в свою очередь, разбивает все множество таких графов на классы эквивалентности - классы изоморфных друг другу графов. Любой описанный выше класс эквивалентности графов называется **непомеченным графом**. В этом смысле обычный простой граф (то есть любой представитель класса эквивалентности) часто называется **помеченным графом**.



Существует четыре различных непомеченных графа на трех вершинах: один отвечает пустому графу, второй – трем помеченным графам с одним ребром, третий – трем помеченным графам с двумя ребрами, и наконец четвертый – полному графу.

Задача перечисления непомеченных графов на n вершинах для произвольного значения параметра n является значительно более сложной задачей по сравнению с задачей перечисления соответствующих помеченных графов.

Характеризация двудольных графов в терминах циклов

Теорема (Кениг)

Граф является двудольным тогда и только тогда, когда в нем отсутствуют циклы нечетной длины (нечетные циклы).

Доказательство.

1. Граф двудольный $\rightarrow$ нет нечетных циклов

Действительно, для того чтобы, выйдя из произвольной вершины x блока X , затем в нее же и вернуться, нам необходимо сделать четное число шагов.

Следовательно, любой цикл в двудольном графе обязан иметь четную длину

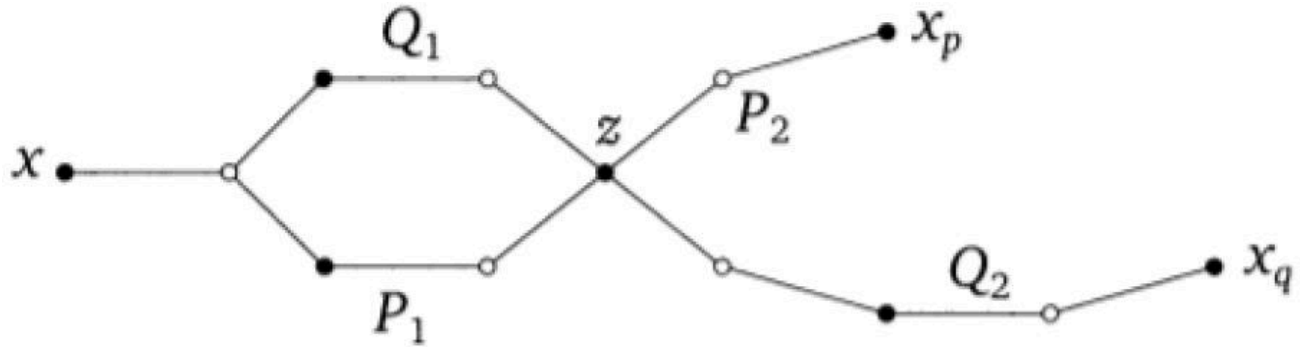
2. Нет нечетных циклов $\rightarrow$ граф двудольный (достаточно ограничиться связными графами, так как любой несвязный граф является двудольным тогда и только тогда, когда двудольной является любая из его компонент связности)

x – произвольная вершина

Блок : все вершины u , длина кратчайшего пути до которых от x нечетная

Блок X : остальные

Докажем, что любые две вершины x, x , из подмножества X смежными не являются.



Рассмотрим произвольные кратчайшие пути P и Q , соединяющие эти вершины с вершиной x .

– последняя общая вершина этих путей.

разделяет пути P и Q на два подпути:

1. участки P_1 и Q_1 от x до z
2. подпути P_2 и Q_2 , соединяющие z с вершинами x_p и x_q соответственно.

Длины путей P_1 и Q_1 обязаны совпадать – в противном случае можно было бы сократить путь от x до одной из двух вершин x, x что невозможно.

Так как x_p и x_q , принадлежат блоку X , длины путей P_2 и Q_2 имеют одинаковую четность, следовательно, одинаковую четность имеют и участки P_2 и Q_2 путей P и Q .

Кроме того, P_2 и Q_2 не имеют никаких других общих вершин, помимо z . Добавление к ним ребра $e = x_p, x_q$ приводит к образованию цикла нечетной длины, чего быть не может.

Аналогично доказывается, что и любые две вершины x_p, x_q блока X смежными не являются. Следовательно, граф является двудольным.

Орграф

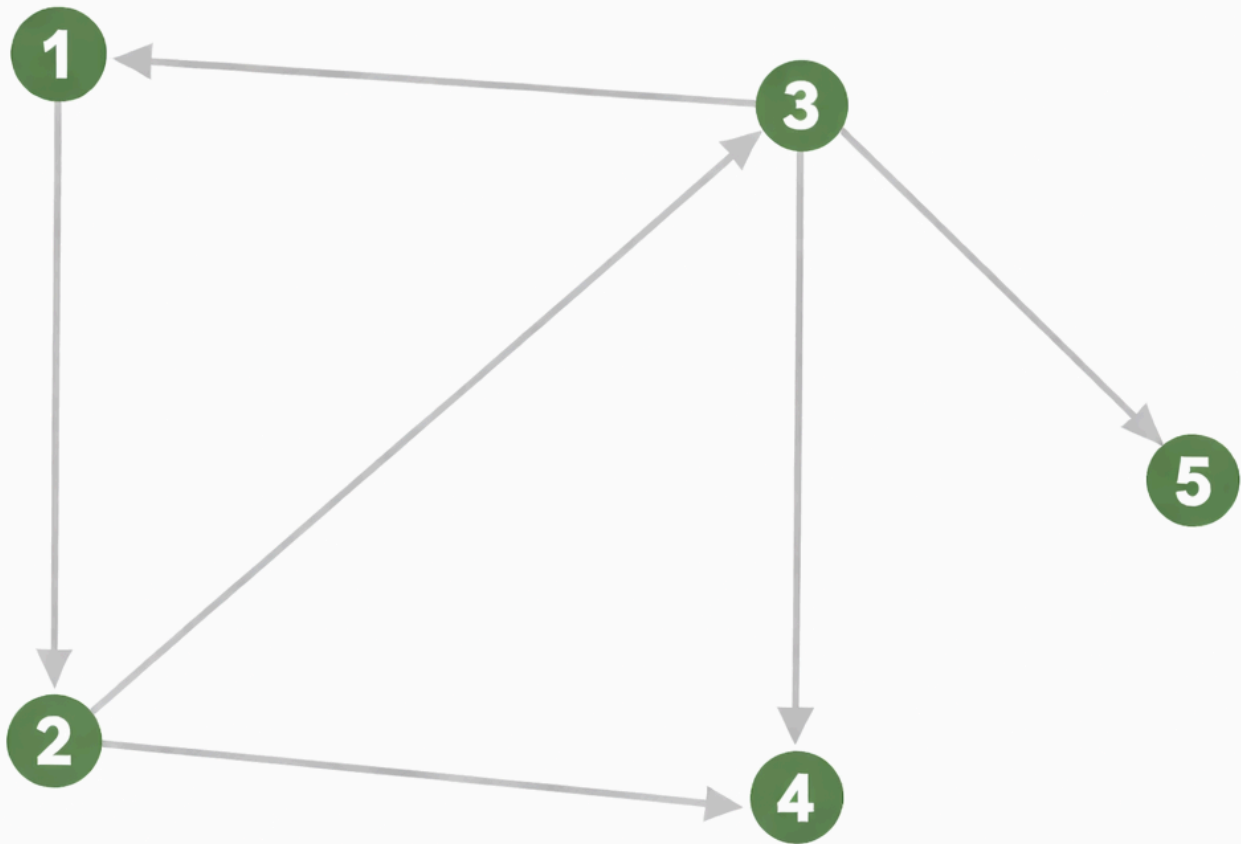
Маршрут, путь, простой путь, цикл

Вершины x и y орграфа называются связанными, если существует хотя бы один путь из x в y и хотя бы один путь из y в x .

Отношение связности – отношение эквивалентности. **Классы эквивалентности** – компоненты сильной связности.

Орграф называется **сильно связным**, если он состоит из единственной компоненты сильной связности.

Орграф называется **слабо связным**, если при замене всех дуг на ребра (неориентированные) получившийся неориентированный граф является связным.



Компоненты этого графа: 1, 2, 3 4 5

Лемма. Пусть C_1 и H две различные компоненты сильной связности орграфа G и пусть существует дуга e из C_1 в H . Тогда дуги из H в C_1 отсутствуют.

По любому орграфу G можно построить так называемый граф $C(G)$ компонент сильной связности графа G , вершинами которого будут компоненты сильной связности графа G , а дугами – дуги графа G , направленные из одной компоненты в другую.

Теорема. В орграфе $C(G)$ циклы отсутствуют, то есть он является **ациклическим орграфом** (DAG – directed acyclic graph).

Доказательство.

Предположим что существует цикл из трех вершин C_1, C_2 и C_3 , то все вершины вовлеченные в этот цикл, находясь в исходном графе, образуют единую компоненту связности, что противоречит нашему условию.

Способы представления графа в компьютере

Матрица смежности

Пусть у нас есть граф $G = (V, E)$, $|V| = n$, $|E| = m$. A : $n \times n$

$$a_{ij} = \begin{cases} 1, & (i, j) \in E \\ 0, & (i, j) \notin E \end{cases}$$

Преимущество: за константу отвечаем на вопрос "Смежны ли данные вершины?"

Недостаток: расходуется много памяти, нужно иметь n^2 ячеек памяти. Если граф разреженный, то большинство этих ячеек будет занято нулями.

Лучше использовать при плотном или небольшом по числу вершин графе.

Список смежности

Когда для каждой вершины существует список смежных с ней вершин.

Преимущество: эффективно используется память, экономим время на обращение

Недостаток: чтобы ответить на вопрос: "Смежны ли данные вершины?", нужно перебрать соответствующий список смежности.

Матрица инцидентности

В этой матрице столбцы соответствуют ребрам, а строки вершинам. В случае неориентированного графа – в столбце соответствующей дуге (i, j) 1 ставят на пересечении строке i и j , а в случае орграфа в столбце дуге (i, j) на пересечении строкой i стоит -1, а j стоит 1. Все другое – это нули.

Недостаток: матрица по большей части содержит нули

Списки инцидентности (ребер)

Списки ребер.

Взвешенный граф

Любому ребру (дуге), соответствует вещественное число, называемое весом, данного ребра или дуги. Используется матрица весов W : $n \times n$.

$$w_{ij} = \begin{cases} \text{вес}(i, j), & (i, j) \in E \\ \infty, & (i, j) \notin E \end{cases}$$

Алгоритмы обходов

Обойти граф – побывать в каждой его вершине. Существует два базовых подхода: обход в ширину и в глубину. В базовом виде алгоритмы обхода казалось бы не несут особой практической пользы, но на самом деле они очень важны, потому что являются основой для множества различных алгоритмов. Существует рекурсивный и нерекурсивный

вариант для обхода в глубину, для алгоритма обхода в ширину обычно используется нерекурсивный вариант. Существует еще два подхода для реализации этих алгоритмов с использованием двух и трех цветов. Если в каких-то случаях все равно, что использовать то бывают случаи когда двух цветов недостаточно.

Обход в глубину

Убираем начальную вершину и проходим от нее вперед настолько насколько можем, то есть находим смежные с ней (непросмотренную) потом смежную с этой (непросмотренная), пока у очередной вершины не окажется смежных (непросмотренных) в этом случае возвращаемся на шаг назад.

Непросмотренные вершины – так как граф является нелинейной структурой, то нам приходится периодически возвращаться и искать новые пути, чтобы не запутаться и не ходить к одному и тому же маршруту, мы должны помечать те места где мы уже были.

Рекурсивный обход в глубину

Массив `used[1:n]` (пройденные вершины)

Граф задается списками смежности `Adj[v]`

До начала работы алгоритма

```
1  for (v in V)
2      used[v] = 0;
```

```
1  void dfs(v)
2      used[v] = 1;
3      for u in Adj[v]`
4          if (!used[u])
5              dfs(u);
```

Для несвязного графа запускается в цикле

```
1  for (v in V)
2      if (!used[v])
3          dfs(v);
```

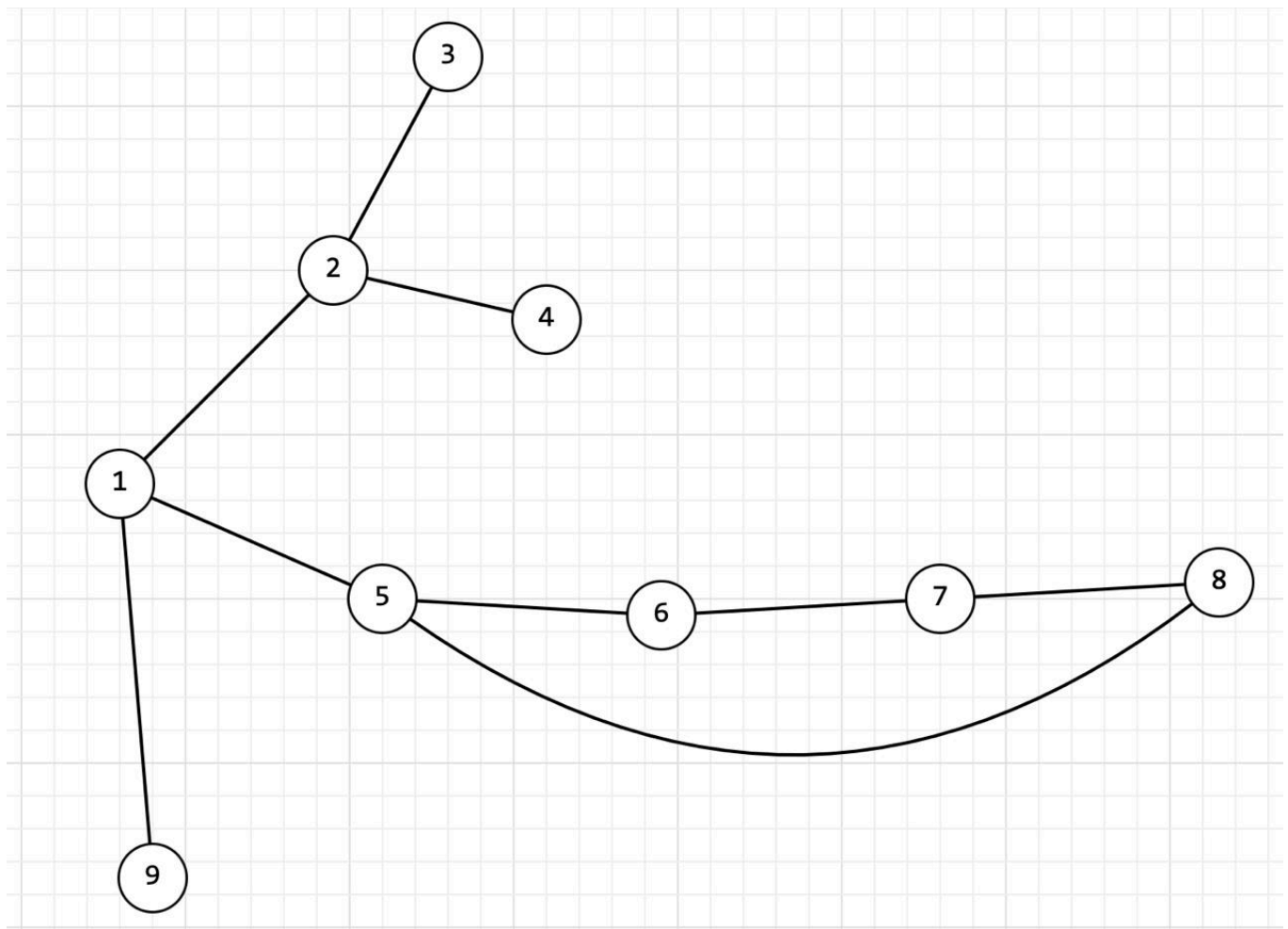
Нерекурсивный обход в глубину (на экзамене)

Для того чтобы перейти к рекурсии к нерекурсии нужно использовать какую-то вспомогательную структуру в которой вы будете хранить вершины к которым вы будете возвращаться в ходе проведения обхода, так как в случае обхода в глубину возвращение всегда идет на шаг назад, то логично использовать стек.

```

1 void dfs(v)
2     used[v] = 1; push(s, v); # s — стек
3     while (!empty(s))
4         top(s, u);
5         if (∃w in Adj[u]: used[w] = 0) # цикл
6             used[w] = 1;
7             push(s, w);
8     else:
9         pop(s);

```



used	стек	экран/файл
000000000	пуст	пуст
100000000	1	1
110000000	21	1, 2
111000000	321	1, 2, 3
- -	21	- -
111100000	421	1, 2, 3, 4

used	стек	экран/файл
- -	21	- -
-	1	- -
111110000	51	1, 2, 3, 4, 5
111111000	651	1, 2, 3, 4, 5, 6
111111100	7651	1, 2, 3, 4, 5, 6, 7
111111110	87651	1, 2, 3, 4, 5, 6, 7, 8
- -	7651	- -
- -	651	- -
- -	51	- -
- -	1	- -
111111111	91	1, 2, 3, 4, 5, 6, 7, 8, 9
- -	1	- -
- -	пуст	- -

Обход в глубину (три цвета)

Обход в глубину (три цвета)

Цвета вершин – белый, серый, черный (массив color)

Метки времени (- вершина и открывается (становится серой), - вершина и закрывается (становится черной))

Массив предшественников [

```

1  DFS(G)
2      for каждой вершины u in V[G]
3          do color[u] ← WHITE
4             π[u] ← NULL
5      time ← 0
6      for каждой вершины u in V[G]
7          do if color[u] = WHITE
8              then DFS_Visit(u)
9  DFS_Visit(u)
10     color[u] ← GRAY // открыта белая вершина u
11     time ← time + 1
12     d[u] ← time
13     for каждой вершины v in Adj[u] // исследование ребра (u, v)
14         do if color[v] = WHITE
15             then π[v] ← u
16                 DFS_Visit(v)

```

```
17     color[u] ← BLACK // завершение
18     f[u] ← time ← time + 1
```

```
1  DFS(G)
2      for каждой вершины u in G.V
3          u.color = WHITE
4          u.π = NULL
5      time = 0
6      for каждой вершины u in G.V
7          if color[u] == WHITE
8              DFS.Visit(G, u)
9  DFC_Visit(G, u)
10     time = time + 1 // открыта белая вершина u
11     u.d = time
12     u.color = GRAY
13     for каждой вершины v in G.Adj[u] // исследование ребра (u, v)
14         if v.color == WHITE
15             v.π = u
16             DFS.Visit(G, v)
17     u.color = BLACK // завершение работы с u
18     time = time + 1
19     u.f = time
```

Подграф предшествования

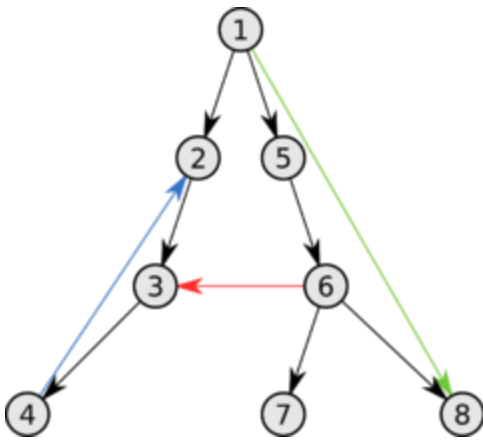
Рассмотрим подграф предшествования обхода в глубину $\mathcal{G} = (V, E)$, $E = \{(u, v) : u, v \in V, [u, v] \in E \text{ и } d[u] < d[v]\}$.

Подграф предшествования поиска в глубину образует лес обхода в глубину, который состоит из нескольких деревьев обхода в глубину. С помощью полученного леса можно классифицировать ребра графа.

Ребрами дерева назовем те ребра из E , которые вошли в $\mathcal{G}$.

Ребра (u, v) , соединяющие вершину u с её предком v в дереве обхода в глубину назовем **обратными ребрами** (для неориентированного графа предок должен быть не родителем, так как иначе ребро будет являться ребром дерева).

Ребра (u, v) , не являющиеся ребрами дерева и соединяющие вершину u с её потомком v в дереве обхода в глубину назовем **прямыми ребрами** (в неориентированном графе нет разницы между прямыми и обратными ребрами, поэтому все такие ребра считаются обратными). Все остальные ребра назовем **перекрестными ребрами** – такие ребра могут соединять вершины одного и того же дерева обхода в глубину, когда ни одна из вершин не является предком другой, или соединять вершины в разных деревьях.



Черным цветом отмечены – ребра дерева, синим – обратные ребра, зеленым – прямые ребра, красным – перекрестные ребра.

Алгоритм dfs можно модифицировать так, что он будет классифицировать встречающиеся при работе ребра. Идея состоит в том, что каждое ребро (u, v) можно классифицировать при помощи цвета вершины u и при первом его исследовании, а именно: Белый цвет вершины по определению dfs говорит о том, что это ребро дерева. Серый цвет в силу того, что серые вершины всегда образуют нисходящий путь в каком-либо из деревьев dfs и встреченная вершина v лежит на нем выше вершины u , определяет обратное ребро (для неориентированного графа необходимо проверить условие $u < v$). Черный цвет, соответственно, указывает на прямое или перекрестное ребро.

Замечание. Можно доказать, что прямые и перекрестные ребра никогда не встречаются при поиске в глубину в неориентированном графе.

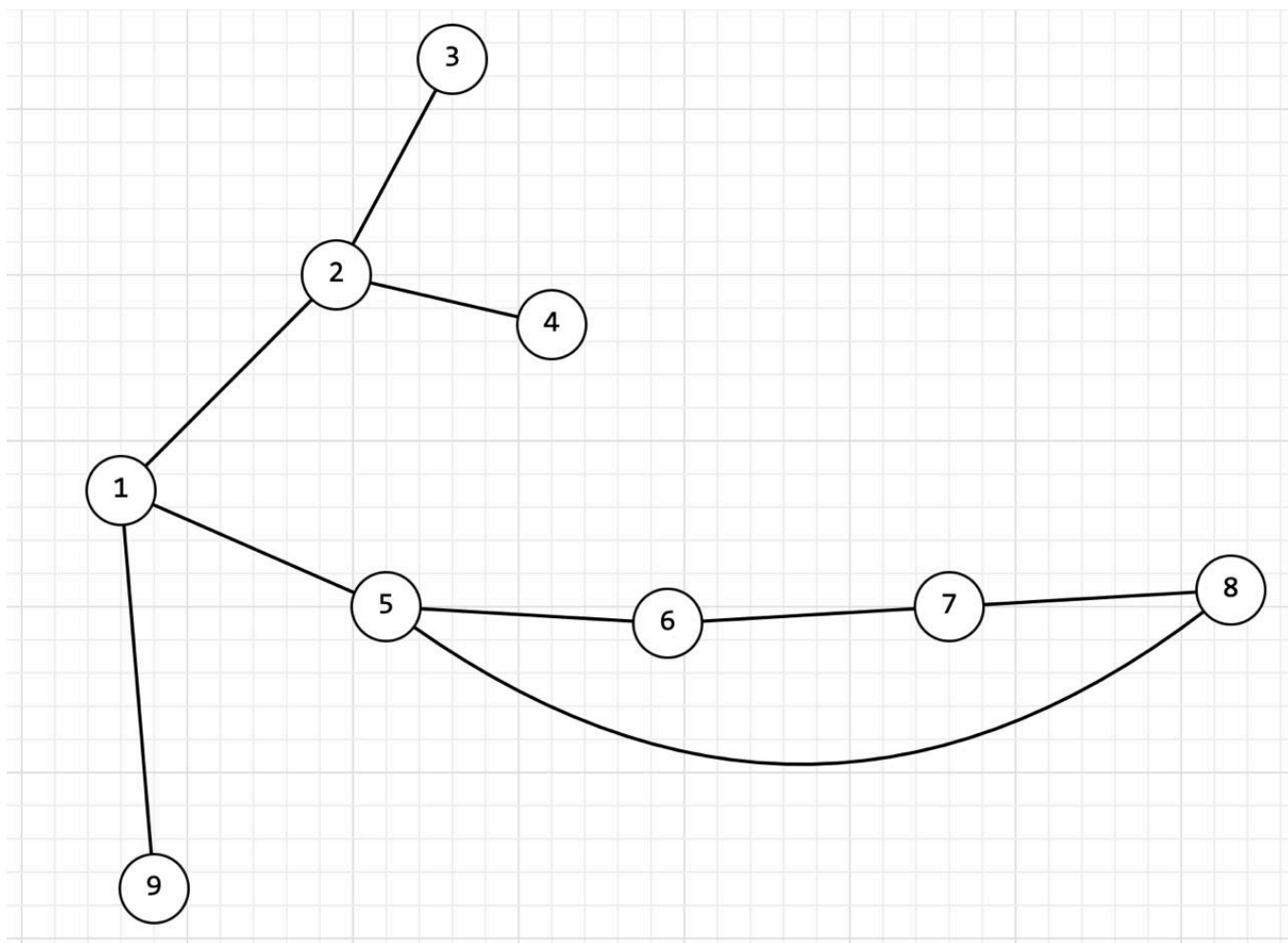
Обход в ширину

Вспомогательная структура – очередь, так как мы обрабатываем сразу все вершины на данном уровне.

```

1 void bfs(v)
2     used[v] = 1; push(q, v)
3     while (!empty(q))
4         pop(q, u)
5         for (w in Adj[u])
6             if (!used[w])
7                 used[w] = 1
8                 push(q, w)

```



used	очередь	экран/файл
000000000	пуста	пуст
100000000	1	1
110010001	259	1, 2, 5, 9
111110001	5924	1, 2, 5, 9, 3, 4
- -	3468	- -
111111111	87	1, 2, 3, 4, 5, 6, 7, 8, 9
- -	7	- -
- -	пуста	- -

Обход в ширину (три цвета)

– предшественник, color – цвет, enqueue – вытащить из очереди, dequeue – добавить в очередь.

```
1 BFS(G, s)
```



```

2      for каждой вершины u in V[G] - s
3          do color[u] ← WHITE
4              d[u] ← ∞
5              π[u] ← NULL
6      color[s] ← GRAY
7      d[s] ← 0
8      π[s] ← NULL
9      Q ← 0
10     Enqueue(Q, s)
11     while Q ≠ 0
12         do u ← Dequeue(Q)
13             for каждой v in Adj[u]
14                 do if color[v] = WHITE
15                     then color[v] ← GRAY
16                         d[v] ← d[u] + 1
17                         π[v] ← u
18                         Enqueue(Q, v)
19     color[u] ← BLACK

```

Алгоритмы, основанные на обходах

Алгоритмы сами по себе ничего не дают, но они очень важны, потому что они являются базой для огромного количества других алгоритмов. Перечислим некоторые из них: поиск (подсчет) компонент связности (неориентированный граф), поиск кратчайших путей (невзвешенный граф), построение остовного дерева/каркаса (невзвешенный граф), проверка графа на ацикличность, топологическая сортировка, поиск множества фундаментальных циклов, проверка графа на двудольность, поиск точек сочленения/мостов, проверка сильной связности.

Подсчет компонент связности

Мы говорили о том что любой обход нужно запускать в таком цикле:

```

1  for (v in V)
2      if (!used[v])
3          dfs(v);

```

Каждый раз когда условие выполняется мы сталкиваемся с очередной компонентой связности, для подсчета просто добавляем счетчик.

```

1  k = 0;
2  for (v in V)
3      if (!used[v])
4          k++;

```

Можно использовать любой обход.

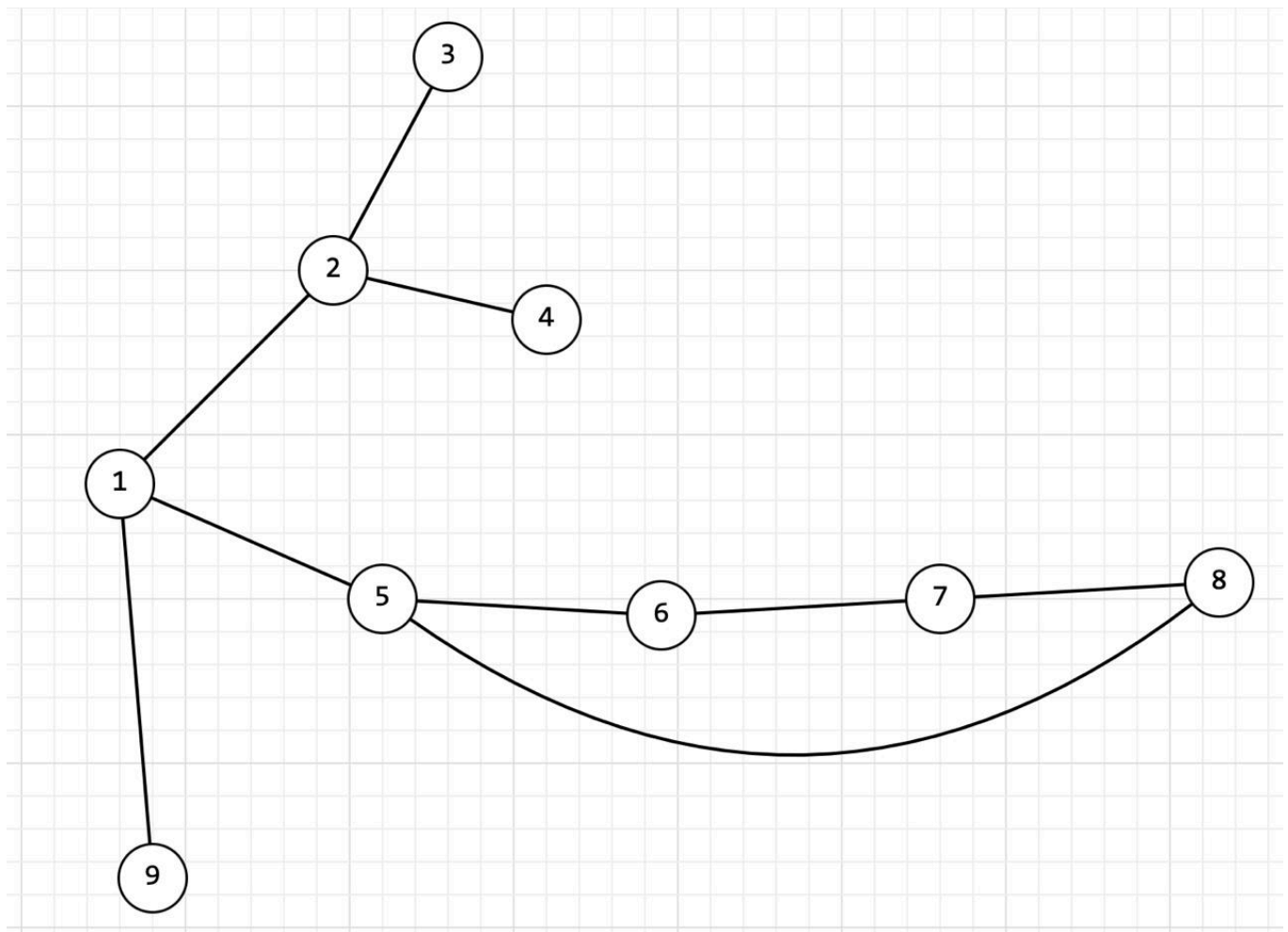
Построение кратчайшего пути (невзвешенный граф)

Заметим сразу, что для того чтобы найти кратчайший путь нужно использовать только обход в ширину. Для того чтобы его использовать нужно иметь массив предков.

```
1 void bfs(v)
2     used[v] = 1
3     π[u] ← NULL
4     push(q, v)
5     while (!empty(q))
6         pop(q, u)
7         for (w in Adj[u])
8             if (!used[w])
9                 used[w] = 1
10                π[w] = u
11                push(q, w)
```

Чтобы получить путь нужно выполнить некие действия:

```
1 print_path(G, s, v)
2     if v == s
3         then print a
4     else if π[v] == NULL
5         then print "Пусть из s в v отсутствует"
6     else print_path(G, s, π[v])
7         print v
```



u	1	2	3	4	5	6	7	8	9
$\pi[u]$	null	1	2	2	1	5	6	5	1

Путь кратчайший: $1 \rightarrow 5 \rightarrow .$

Построение остовного дерева/каркаса (невзвешенный граф)

Дерево – это связный граф, не содержащий циклов. Если остовной подграф является дерево, то он является каркасом.

$= (,)$

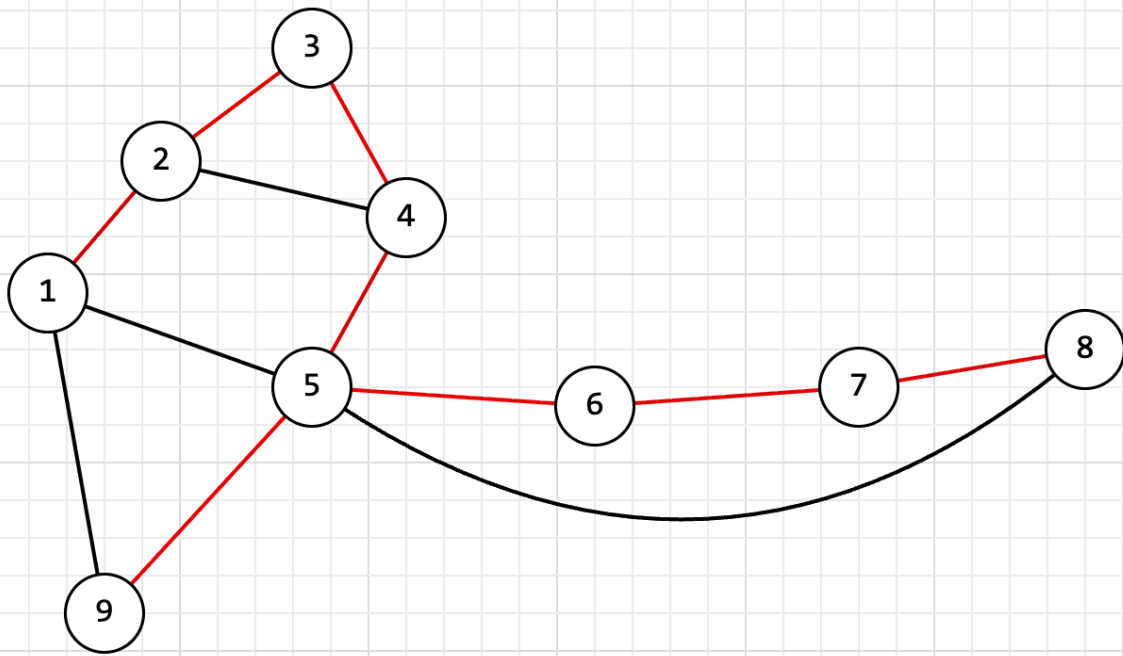
$= (, _2)$ – дерево

–

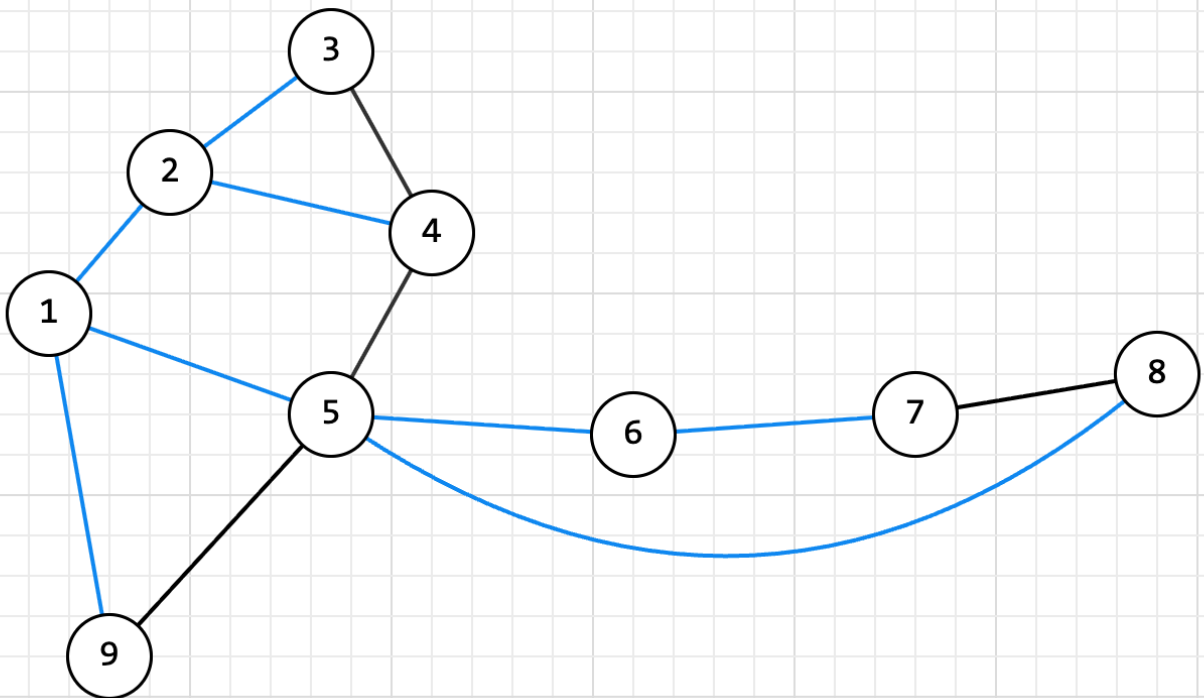
$= (, w)$

Подходит любой алгоритм обхода, но каркасы будут разные.

Обход в глубину:



Обход в ширину:



Топологическая сортировка

Топологическая сортировка (topological sort) ориентированного ациклического графа (directed acyclic graph – "dag") = (,) представляет собой упорядочивание вершин таким образом, что для любой дуги (,) () номер вершины меньше номера вершины .

```

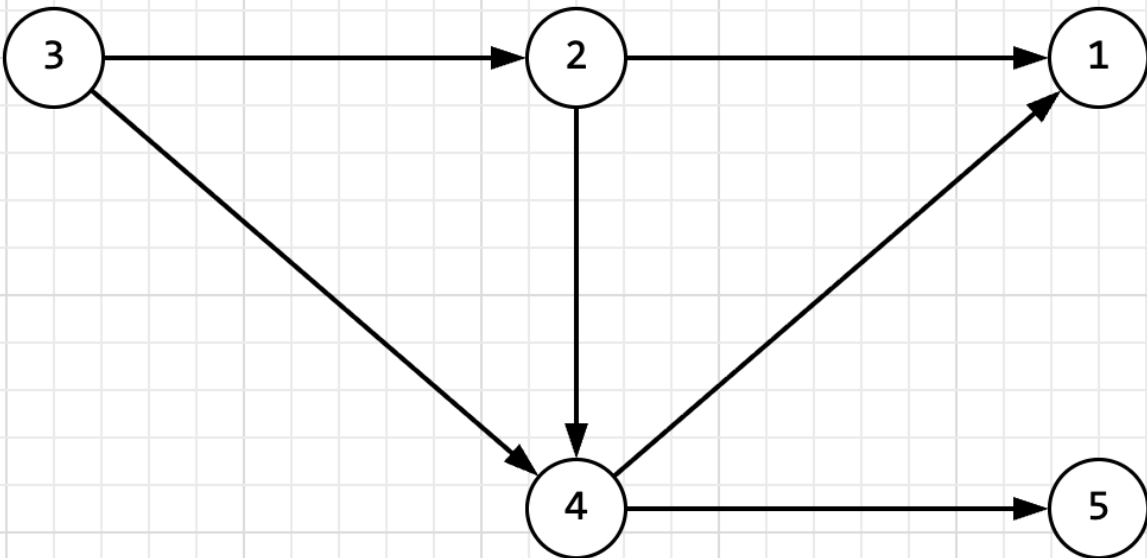
1  topological_sort(G)
2      Вызов DFS(G) для вычисления времени завершения f[v]
3      для каждой вершины v

```

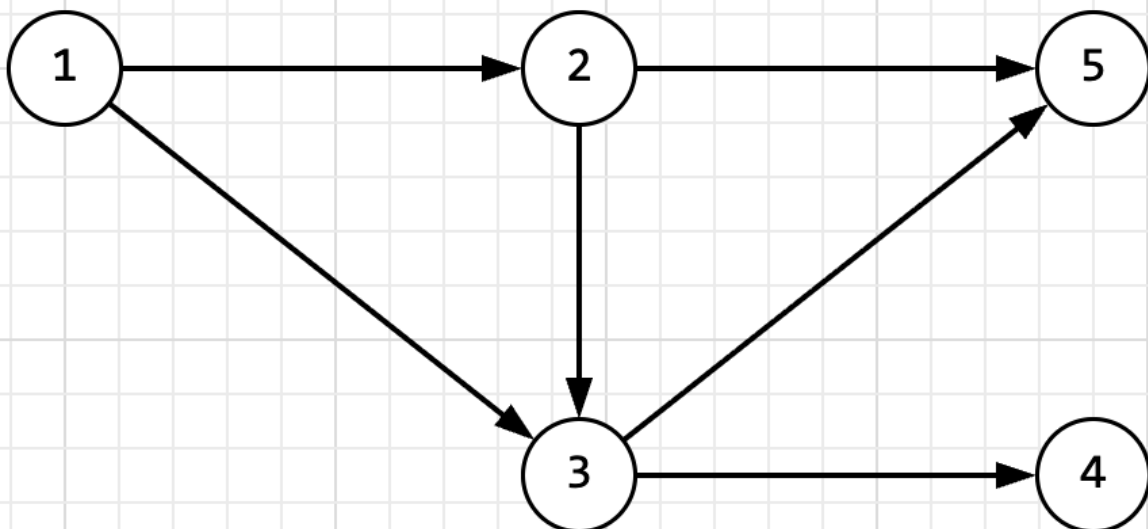
4

По завершении работы над вершиной внести ее в начало связанного списка

5

return Связанный список вершин

Обход в глубину из 3 вершины: 1, 5, 4, 2, 3



Проверка на ацикличность

В случае **ориентированного графа** произведём серию обходов в глубину.

То есть из каждой вершины, в которую мы ещё ни разу не приходили, запустим поиск в глубину, который при входе в вершину будет красить её в серый цвет, а при выходе из нее – в чёрный. И, если алгоритм пытается пойти в серую вершину, то это означает, что цикл найден.

В случае **неориентированного графа**, необходимо дополнительно проверять, что

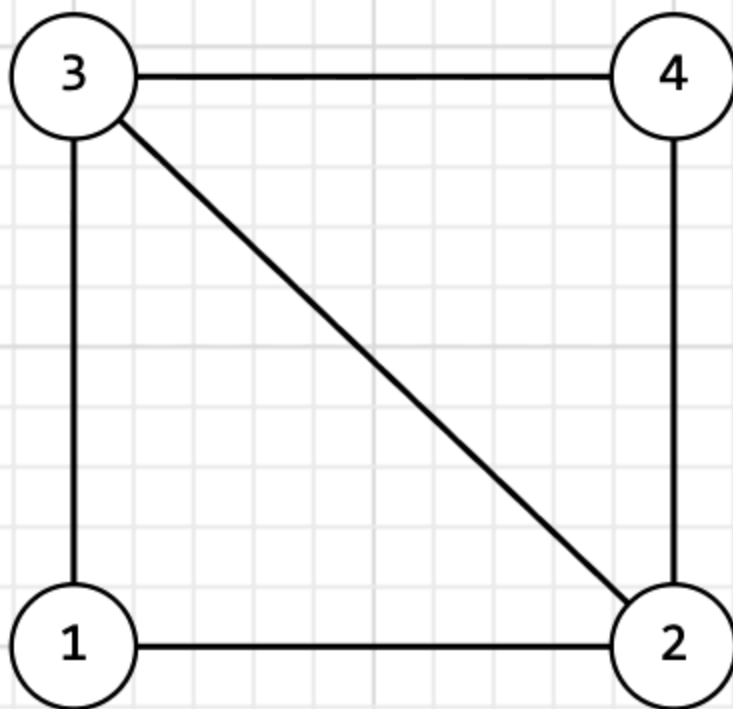
текущее рассматриваемое из вершины ребро не является тем ребром, по которому мы пришли в эту вершину.

Для **восстановления** самого цикла достаточно при запуске поиска в глубину из очередной вершины добавлять эту вершину в стек. Когда поиск в глубину нашел вершину, которая лежит в цикле, будем последовательно вынимать вершины из стека, пока не встретим найденную еще раз.

Реализация для случая ориентированного графа

```
1  // color – массив цветов, изначально все вершины белые
2  void dfs(v) {
3      color[v] = grey;
4      for u in Adj[v] do // Adj – список смежности вершины v
5          if (color[u] == white)
6              dfs(u)
7          if (color[u] == grey)
8              circle = true
9      color[v] = black
10 }
11 for v in V do
12     color[v] = white;
13 circle = false
14 for v in V do
15     if color[v] == white
16         dfs(v)
```

Можно ли найти все циклы? Смысла искать все циклы нет. Например на рисунке 3 цикла, но достаточно найти 2, а 3 построить на их основе.



Вводим понятие фундаментальных циклов графа – это множество таких циклов, что с помощью их линейной комбинации можно выразить все циклы.

Алгоритм. (Нахождение множества элементарных циклов графа)

Данные: Граф = (V, E) , представленный списками инцидентности ЗАПИСЬ[v],

Результаты: Множество элементарных циклов графа.

```

1  procedure ЦИКЛ(u): (* нахождение фундаментального множества циклов для
    компоненты связности, содержащей вершину u, переменные d, num, СТЕК,
    ЗАПИСЬ, WGN – глобальные*)
2      begin d := d + 1; СТЕК[d] := v; num := num + 1; WGN[v] := num;
3          for u in ЗАПИСЬ[v] do
4              if WGN[u] = 0 then ЦИКЛ(u)
5              else if (u != СТЕК[d - 1]) and (WGN[v] > WGN[u]) then (* {v,
    u} замыкает новый цикл *)
6                  выписать цикл с вершинами
7                  СТЕК[d], СТЕК[d - 1], ..., СТЕК[e], где СТЕК[e] = u;
8                  d := d - 1 (* использованная вершина v удаляется из стека *)
9          end; (* ЦИКЛ *)
10     begin (* главная программа *)
11         for v in V do WGN[v] := 0; num := 0; (* инициализация *)
12         d := 0; СТЕК[0] := 0; (* d = число элементов в стеке *)
13         for v in V do
14             if WGN[v] = 0 then ЦИКЛ(u)
  
```


Оценим теперь вычислительную сложность этого алгоритма. Отметим сначала, что общее число шагов, не считая выписывания циклов (строки 6, 7) — как и во всех алгоритмах, основанных на поиске в глубину, — имеет порядок (n) . К этому следует прибавить суммарную длину всех циклов. Эта длина не превосходит $(-n - 1)$, что дает общую сложность алгоритма $(n - n)$, (или (n) , если отбросить вырожденный случай $= 0$).

Проверка графа на двудольность

На основе обхода в глубину

На каждом шаге обхода в глубину помечаем вершину.

Допустим, мы пошли в первую вершину — помечаем её как 1. Затем просматриваем все смежные вершины, и если не помечена вершина, то на ней ставим пометку 2 и рекурсивно переходим в нее. Если же она помечена и на ней стоит та же пометка, что и у той, из которой шли (в нашем случае 1), значит граф не двудольный.

```

1  void dfs(v)
2      for u in Adj[v] do
3          if color[u] == 0
4              color[u] = 3 - color[v]
5              dfs[u]
6          else if color[u] == color[v]
7              flag = false
8
9  for (v in V) do
10     color[v] = 0;
11     flag = true
12     for v in V do
13         if color[v] == 0
14             color[v] = 1
15             dfs[v]
```

На основе обхода в ширину

Произведём серию поисков в ширину. Т.е. будем запускать поиск в ширину из каждой непосещённой вершины. Ту вершину, из которой мы начинаем идти, мы помещаем в первую долю. В процессе поиска в ширину, если мы идём в какую-то новую вершину, то мы помещаем её в долю, отличную от доли текущей вершины. Если же мы пытаемся пройти по ребру в вершину, которая уже посещена, то мы проверяем, чтобы эта вершина и текущая вершина находились в разных долях. В противном случае граф двудольным не

является. По окончании работы алгоритма мы либо обнаружим, что граф не двудольный, либо найдём разбиение вершин графа на две доли.

Поиск кратчайших путей во взвешенном графе

Граф называется **взвешенным**, если каждому его ребру (дуге) приписано некоторое вещественное число, называемое весом данного ребра (дуги).

Кратчайший путь – это путь минимального веса.

Вес пути – это сумма весов, входящих в него ребер (дуг).

Существует множество прикладных задач, которые можно решать с помощью данных алгоритмов, причем не обязательно вес – это именно длина пути в км (или других единицах измерения длин). Это может быть и цена бензина, цена билета (на поезд или самолет) или вообще любая стоимость проезда и т.д.

В основном все известные алгоритмы решают один из двух вариантов задачи:

1. поиск кратчайшего пути от заданной вершины до всех остальных (алгоритм Форда, Дейкстры, алгоритм поиска кратчайших путей в ориентированном ациклическом графе)
2. поиск кратчайшего пути между всеми парами вершин (алгоритм Флойда, алгоритм Джонсона)

Кроме того, у этих алгоритмов могут различаться требования к графу, с которым они работают и время работы. Например, алгоритм Дейкстры работает только с графами, у которых все веса положительны.

Еще на выбор алгоритма может повлиять такое свойство графа как **разреженность** (например, на разреженных графах алгоритм Джонсона будет работать быстрее, чем алгоритм Флойда)

Базовым для всех рассматриваемых алгоритмов является так называемое соотношение релаксации.

Будем вес дуги (u, v) обозначать как $w(u, v)$, а расстояния от выбранной и зафиксированной вершины u до произвольной вершины v будем обозначать $d[u, v]$.

Инициализация и соотношение релаксации

```
1  initialize_single_source(G, s)
2      for всех v in V(G)
3          do d[v] ← ∞
4      d[s] ← 0
5
6  relax(u, v, w)
7      if d[v] > d[u] + w(u, v)
8          then d[v] ← d[u] + w(u, v)
```

Алгоритм Беллмана-Форда

Алгоритм Беллмана-Форда решает задачу о кратчайшем пути в общем случае, когда веса ребер могут быть отрицательными.

Алгоритм возвращает значение TRUE, если путь найден и FALSE – если это сделать невозможно в связи с наличием цикла отрицательного веса.

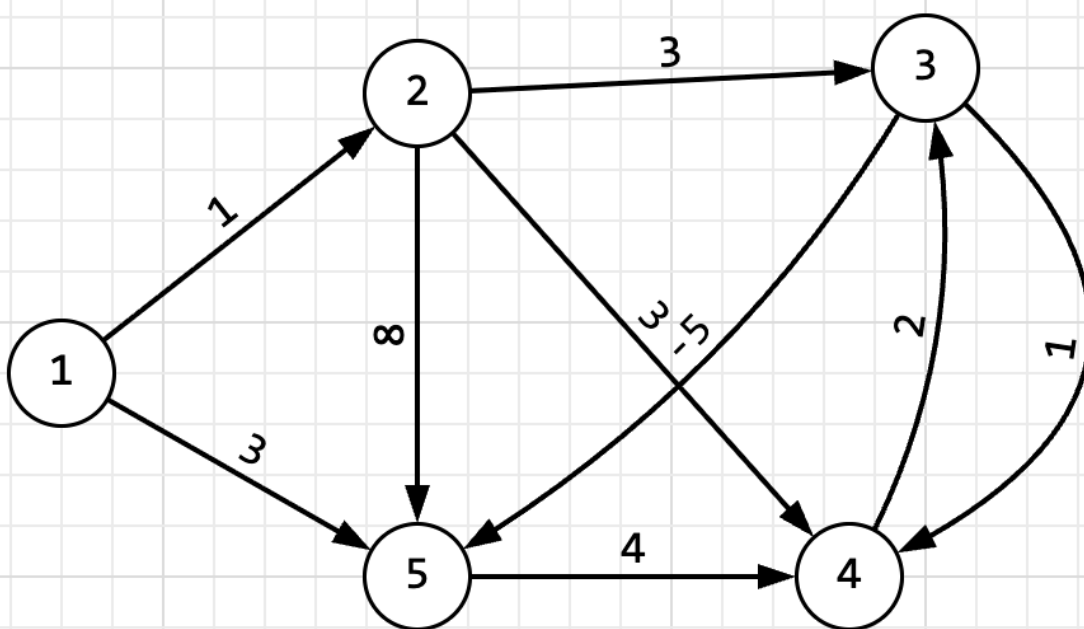
В том случае, если кратчайшие пути существуют, алгоритм возвращает их **длины**.

```
1  bellman_ford(G, w, s)
2      initialize_single_source(G, s)
3      for i ← 1 to |V[G]| - 1
4          do for каждого ребра (u, v) in E[G]
5              do Relax(u, v, w)
6      for каждого ребра (u, v) in E[G]
7          do if d[v] > d[u] + w(u, v)
8              then return FALSE
9      return TRUE
```

Шаги 3-5 заполняют массив кратчайших путей, на случай цикла отрицательного веса 6-8 шаги.

Время выполнения ($O(|V|^3)$).

Пример



Матрица смежности

	1	2	3	4	5
1	∞	1	∞	∞	3
2	∞	∞	3	3	8
3	∞	∞	∞	1	-5
4	∞	∞	2	∞	∞
5	∞	∞	∞	4	∞

Матрица длин

	d1	d2	d3	d4	d5
1	0	∞	∞	∞	∞
2	0	1	∞	∞	3
3	0	1	4	4	-1
4	0	1	4	3	-1
5	0	1	4	3	-1

А как восстановить сами пути? Для этого нужно сохранять предшественников (parents) на каждом шаге релаксации.

Алгоритм Дейкстры

В алгоритме Дейкстры предполагается, что все веса положительны.

В алгоритме Дейкстры создается множество вершин, для которых уже вычислены окончательные веса кратчайших путей к ним (от выделенной вершины).

По ходу алгоритма поочередно выбирается такая вершина и из множества —, которой на данном этапе соответствует минимальная оценка кратчайшего пути.

Данная вершина добавляется в множество и производится релаксация всех исходящих из нее ребер.

В приведенной далее реализации используется неубывающая очередь с приоритетами, состоящая из вершин, в роли ключей для которых выступают значения.

```

1  dijkstra(G, w, s)
2      initialize_single_source(G, s)
3      S ← ∅
4      Q ← V[G]
5      while Q != ∅
6          do u ← extract_min(Q)

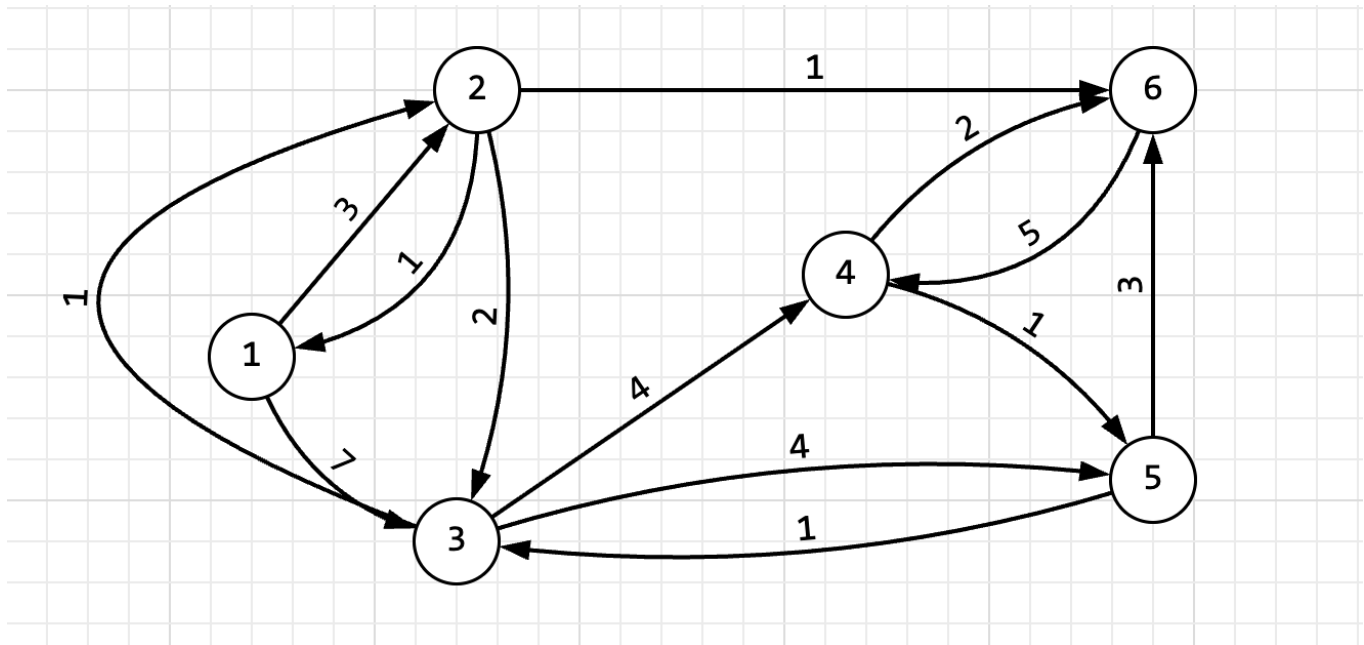
```

```

7         s ← s ∪ {u}
8         for все v in Adj[u]
9             do relax(u, v, w)

```

Пример



n	d1	d2	d3	d4	d5	d6
1	0	3	7	∞	∞	∞
2	0	3	5	∞	∞	4
3	0	3	5	9	∞	4
4	0	3	5	9	9	4
5	0	3	5	9	9	4
6	0	3	5	9	9	4

Ориентированный ациклический граф

Проведем топологическую сортировку его вершин. Тогда, если последовательно проводить релаксацию в полученном с помощью топологической сортировки порядке (вершин), то можно найти кратчайшие пути из выделенной вершины. до всех остальных за время ().

```

1 dag_shortest_paths(G, w, s)
2     топологическая сортировка вершин графа G
3     initialize_single_source(G, s)
4     for каждой вершины u в порядке топологической сортировки

```

```

5         for v in Adj[u]
6             relax(u, v, w)

```

Кратчайшие пути между всеми парами вершин

Задачу о поиске кратчайших путей между всеми парами вершин можно решить, n раз запустив алгоритм поиска кратчайших путей из единого истока, каждый раз выбирая в качестве истока новую вершину графа. Если веса всех ребер неотрицательные, можно воспользоваться алгоритмом Дейкстры. Если используется реализация неубывающей очереди с приоритетами в виде линейного массива, то время работы такого алгоритма равно (n^3) .

Если допускается наличие ребер с отрицательным весом, алгоритм Дейкстры неприменим. Вместо него для каждой вершины следует выполнить алгоритм Беллмана-Форда, который работает медленнее. Полученное в результате время работы будет равно (n^3) , которое на плотных графах можно записать как (n^4) .

В качестве входных данных будем использовать матрицу w_{ij} размером $n \times n$ (наличие дуг отрицательного веса допускается, но предполагается, что нет циклов отрицательного веса)

$$w_{ij} = 0 \quad \text{если } i = j, \text{ вес}(i, j) \quad \text{если } i \neq j, \infty \quad \text{если } i \neq j \text{ и нет ребра } (i, j).$$

Выходные данные будем представлять в виде матрицы π_{ij} размером $n \times n$, содержащую длины кратчайших путей между всеми парами вершин. Для восстановления путей нам понадобится матрица предшествования.

π_{ij} , где величина π_{ij} имеет значение NIL, если $i = j$ или путь из вершины i в вершину j отсутствует; в противном случае π_{ij} - предшественник вершины j на некотором кратчайшем пути из вершины i .

```

1  print_all_pairs_shortest_path(П, i, j)
2      if i = j
3          then print i
4      else if  $\pi_{ij}$  = NULL
5          then print "Не существует пути из i в j"
6      else print_all_pairs_shortest_path(П, i,  $\pi_{ij}$ )
7          print j

```

Лемма. Частичные пути кратчайшего пути являются кратчайшими путями

Пусть $p = (v_1, v_2, \dots, v_n)$ — кратчайший путь из вершины v_1 к вершине v_n в заданном взвешенном ориентированном графе $G = (V, E)$ с весовой функцией $w : E \rightarrow \mathbb{R}$, а $p_{ij} = (v_i, v_{i+1}, \dots, v_j)$ — частичный путь пути p , который проходит из вершины v_i к вершине v_j для произвольных i и j , удовлетворяющих неравенству $1 \leq i \leq j \leq n$. Тогда p_{ij} — кратчайший путь из вершины v_i к вершине v_j .

Доказательство. Если разложить путь p на составные части p_{1i}, p_{ij}, p_{jn} , то будет выполняться соотношение $w(p) = w(p_{1i}) + w(p_{ij}) + w(p_{jn})$. Теперь предположим, что существует путь q_{ij} из вершины v_i к вершине v_j , вес которого удовлетворяет неравенству $w(q_{ij}) < w(p_{ij})$. Тогда $q = p_{1i} \cup q_{ij} \cup p_{jn}$ — путь из вершины v_1 к вершине v_n , вес которого $w(q) = w(p_{1i}) + w(q_{ij}) + w(p_{jn})$ меньше веса $w(p)$. Это противоречит предположению о том, что p — кратчайший путь из вершины v_1 к вершине v_n .

Алгоритм Флойда

Пусть $d_{ij}^{(0)}$ — вес кратчайшего пути из вершины v_i в вершину v_j , для которого все промежуточные вершины принадлежат множеству $\{v_1, v_2, \dots, v_{i-1}\}$. Если $d_{ij}^{(0)} = 0$, то путь из вершины v_i в вершину v_j , в котором отсутствуют промежуточные вершины с номером, большим нуля, не содержит промежуточных вершин вообще. Такой путь содержит не более одного ребра, поэтому $d_{ij}^{(0)} = w_{ij}$. Рекурсивное определение, которое соответствует приведенному выше описанию, дается соотношением

$$d_{ij}^{(k)} = w_{ij} \quad \text{если } k = 0, \quad \left(d_{ij}^{(k-1)}, d_{ik}^{(k-1)}, d_{kj}^{(k-1)} \right) \quad \text{если } k \geq 1.$$

Поскольку все промежуточные вершины произвольного пути принадлежат множеству $v_1, v_2, \dots, v_n$, матрица $d_{ij}^{(n)} = (d_{ij}^{(n)})$ дает конечный ответ: $d_{ij}^{(n)} = d_{ij}$ для всех пар вершин v_i, v_j . Предположим, что граф состоит из вершин $v_1, v_2, \dots, v_n$. Рассмотрим подмножество вершин $v_1, v_2, \dots, v_{i-1}$ для некоторого i . Для произвольной пары вершин v_j, v_i рассмотрим все пути из вершины v_i в вершину v_j , все промежуточные вершины которых выбраны из множества $\{v_1, v_2, \dots, v_{i-1}\}$. Пусть среди этих путей p — путь с минимальным весом (этот путь простой). Тогда:

1. если v_i не промежуточная вершина пути p , то все промежуточные вершины этого пути будут принадлежать множеству $\{v_1, v_2, \dots, v_{i-1}\}$. Таким образом, кратчайший путь из вершины v_i в вершину v_j со всеми промежуточными вершинами из множества $\{v_1, v_2, \dots, v_{i-1}\}$ одновременно является кратчайшим путем из вершины v_i в вершину v_j со всеми промежуточными вершинами из множества $\{v_1, v_2, \dots, v_n\}$.
2. если v_i — промежуточная вершина пути p , то его можно разбить на два пути: p_1 (между v_i и v_{i+1}) и p_2 (между v_{i+1} и v_j).

Согласно лемме p_1 — кратчайший путь из v_i в v_{i+1} , все промежуточные вершины которого принадлежат множеству $\{v_1, v_2, \dots, v_{i-1}\}$. Но так как v_i не является промежуточной вершиной пути p_1 , понятно, что p_1 — кратчайший путь из v_i в v_{i+1} , все промежуточные вершины которого

принадлежат множеству $(1, 2, \dots, k-1)$. Аналогично p_2 – кратчайший путь из k в j , все промежуточные вершины которого принадлежат множеству $(1, 2, \dots, k-1)$.

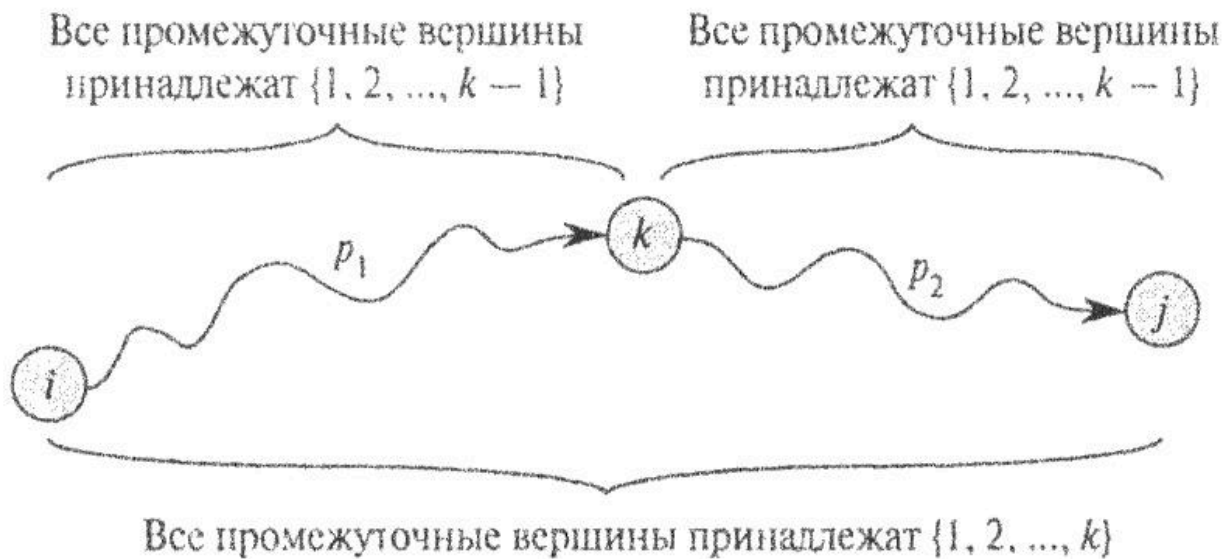


Рис. 25.3. Схема, иллюстрирующая структуру кратчайших путей

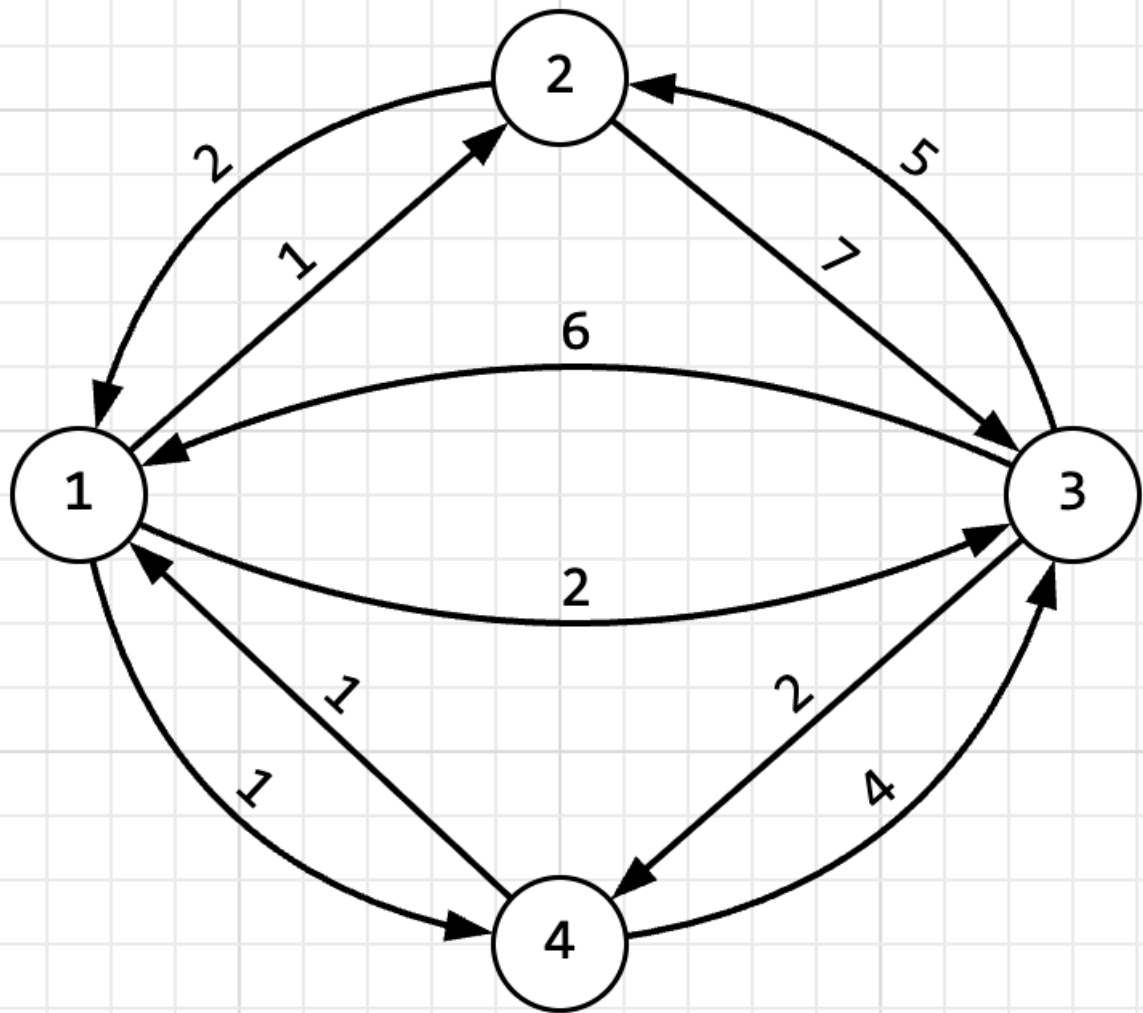
В качестве входных данных выступает матрица размерами $n \times n$. Процедура возвращает матрицу $D^{(n)}$, содержащую веса кратчайших путей.

```

1  floyd_warshall(W)
2      n = rows[W]
3      D_0 = W
4      for k = 1 to n
5          do for i = 1 to n
6              do for j = 1 to n
7                  do d_ij_k = min(d_ij_k_1, d_ik_k_1 + d_kj_k_1) // формула
8  выше
9      return D_n

```

Пример



(0)

	1	2	3	4
1	0	1	2	1
2	2	0	7	∞
3	6	5	0	2
4	1	∞	4	0

(1)

```

| 1 | 2 | 3 | 4 | |
|---|---|---|---|---|
| 1 | 0 | 1 | 2 | 1 |
| 2 | 2 | 0 | 4 | 3 |
| 3 | 6 | 5 | 0 | 2 |
| 4 | 1 | 2 | 3 | 0 |

```

(2)

```

| 1 | 2 | 3 | 4 | |
|---|---|---|---|---|
| 1 | 0 | 1 | 2 | 1 |
| 2 | 2 | 0 | 4 | 3 |
| 3 | 6 | 5 | 0 | 2 |
| 4 | 1 | 2 | 3 | 0 |

```

(3)

```

| 1 | 2 | 3 | 4 | |
|---|---|---|---|---|
| 1 | 0 | 1 | 2 | 1 |
| 2 | 2 | 0 | 4 | 3 |
| 3 | 6 | 5 | 0 | 2 |
| 4 | 1 | 2 | 3 | 0 |

```

(4)

```

| 1 | 2 | 3 | 4 | |
|---|---|---|---|---|
| 1 | 0 | 1 | 2 | 1 |
| 2 | 2 | 0 | 4 | 3 |
| 3 | 3 | 4 | 0 | 2 |
| 4 | 1 | 2 | 3 | 0 |

```

Деревья

Дерево – это простой связный граф без циклов.

Простой (не обязательно связный) граф без циклов называется **лесом**.

Вершина дерева, имеющая степень 1, называется **листом**.

Лемма. У любого дерева построенного на $n \geq 2$ вершинах, имеется как минимум 2 листа.

Доказательство.

Рассмотрим произвольный простой путь $= (x_1, x_2, \dots, x_k)$ максимальной длины. Его концы – листья (докажем от противного). Пусть x_1 не лист. Тогда у x_1 есть еще одно инцидентное ребро. Может быть 2 случая:

1. Это ребро приходит в какую-то вершину пути. Тогда имеем цикл. Противоречие.
2. Это ребро приходит не в вершину пути. Тогда имеем путь, у которого длина больше k . Противоречие.

Теорема. В дереве, построенном на n вершинах, имеется ровно $n - 1$ ребро.

Доказательство по индукции (по количеству вершин).

1. Дерево, состоящее из $n = 1$ вершины, не имеет ни одного ребра.
2. Предположим, что утверждение доказано для деревьев, построенных на n вершинах.
3. Докажем для произвольного дерева с $n + 1$ вершиной.

По доказанной выше лемме у любого такого дерева имеется хотя бы один лист x .

Удалим этот лист x вместе с инцидентным ему ребром e . Полученный в результате такой операции граф :

а) останется связным. Действительно, любой путь, соединяющий в произвольные две вершины, отличные от x , ребро e содержать не может. Поэтому удаление e на связность оставшихся в T вершин никак не повлияет.

б) не приобретет циклов

Следовательно, граф T является деревом, построенным на n вершинах. По предположению индукции у него $n - 1$ ребро. Тогда у исходного графа n ребер.

Обратная теорема. Любой простой связный граф, построенный на n вершинах и имеющий $n - 1$ ребро, является **деревом**.

Доказательство от противного.

Существует цикл C .

Любое ребро e , принадлежащее циклу C , мостом в графе не является (характеризация моста), то есть его удаление не приводит к потере связности.

Будем удалять по очереди ребра из циклов, пока они есть, получим в итоге простой связный граф без циклов, то есть дерево. По предыдущей теореме у него $n - 1$ ребро.

Следствие. Всякий связный граф, построенный на n вершинах, имеет как минимум $n - 1$ ребро.

Доказательство аналогично предыдущему.

По-другому означает, что у всякого связного графа существует остовное дерево.

Утверждение. Связный граф является деревом тогда и только тогда, когда любое его ребро является мостом.

Доказательство.

По определению в дереве T циклы отсутствуют. Следовательно, ни одно из его ребер циклу не принадлежит. Согласно основной характеристике моста это означает, что любое ребро графа T является мостом. **Обратно**, если любое ребро связного графа является мостом, то оно не принадлежит никакому циклу. Поэтому данный граф обязан быть деревом.

Утверждение. Граф является деревом тогда и только тогда, когда для любых его вершин существует единственный простой путь, соединяющий эти вершины.

Доказательство.

Пусть в T для любой пары вершин существует единственный соединяющий их простой путь. Тогда T является связным графом. Пусть в таком графе существует цикл C . Тогда между любыми двумя вершинами этого цикла существует два различных пути, соединяющие эти вершины. Противоречие.

Обратно. Пусть T является деревом, и пусть в нем существуют два различных простых пути P и Q , соединяющие какие-то две вершины x и y . Тогда имеем цикл. Противоречие.

Следствие. Пусть T есть остовное дерево связного графа, и пусть e - ребро, не принадлежащее T . Тогда граф $T + e$ содержит единственный цикл, который проходит через e .

Доказательство.

Любой цикл в графе $T + e$ обязан содержать ребро $e = x, y$. Если это не так, то мы имеем цикл в дереве T . Кроме того, C есть цикл в $T + e$ тогда и только тогда, когда $C - e$ — это путь от x к y в исходном дереве T . Но в дереве T путь от x к y единствен. Следовательно, $T + e$ содержит единственный цикл, проходящий через ребро e .

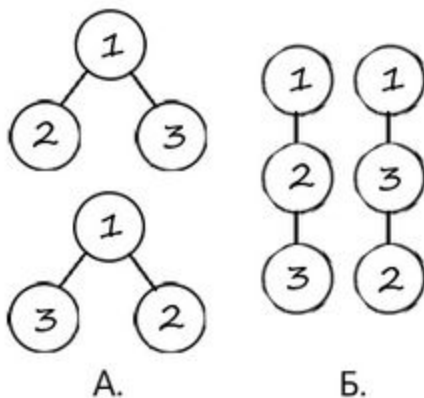
Корневым деревом называется дерево с выделенной вершиной, называемой **корнем**.

Корневым лесом называется лес, каждая компонента связного которого представляет собой корневое дерево.

Отец, сын, предок, потомок, высота дерева, -арное дерево (например, бинарное).

Невозрастающая последовательность неотрицательных чисел $d_1, d_2, \dots, d_n, 1, 2, \dots, n, 0$, называется **графовой**, если она является последовательностью степеней вершин некоторого простого графа.

Перечисление всех помеченных деревьев, построенных на n вершинах



Теорема (формула Кэли)

Количество n , различных помеченных деревьев на n вершинах равно n^{n-2}

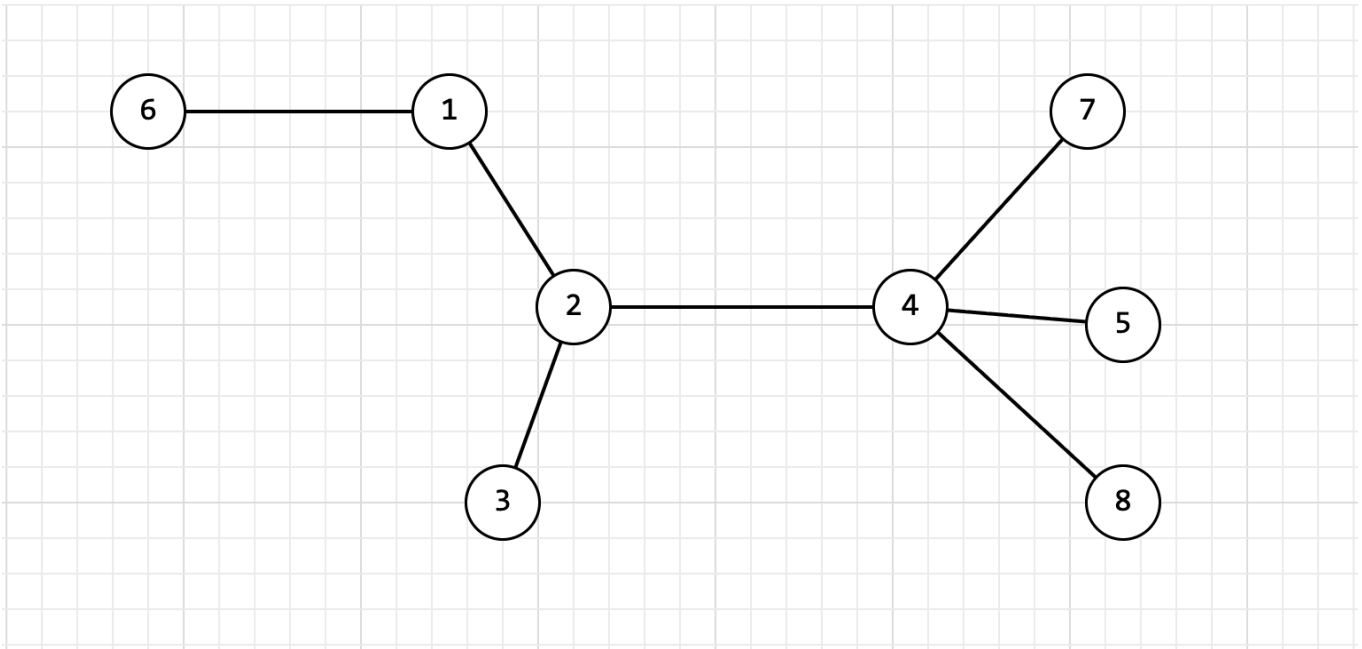
Доказательство (Х.Прюфер, 1918 г.)

Код Прюфера или последовательность Прюфера:

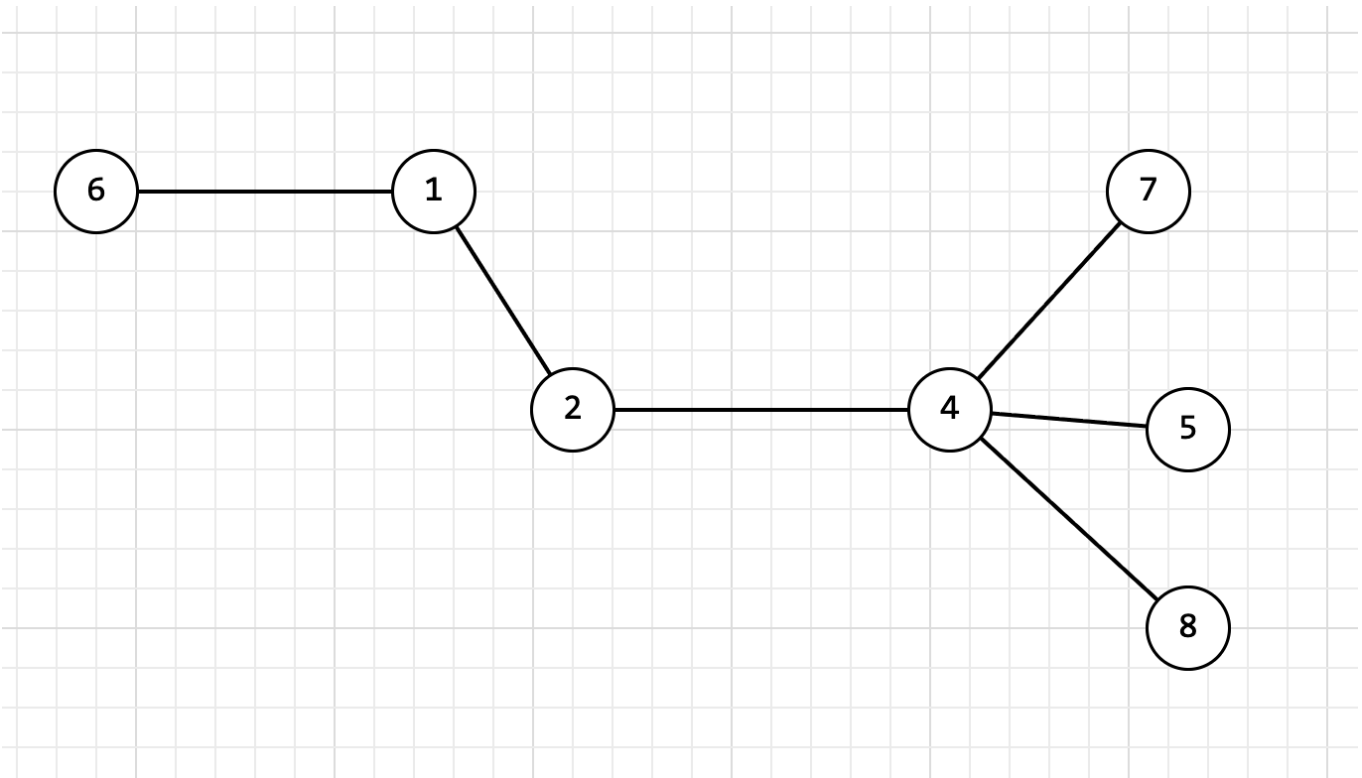
Переводит помеченные деревья порядка n в последовательности чисел по следующему алгоритму:

1. Выбирается лист с минимальным номером.
2. Лист и инцидентное ему ребро удаляются из дерева, а в последовательность Прюфера добавляется номер смежной с данным листом вершины.

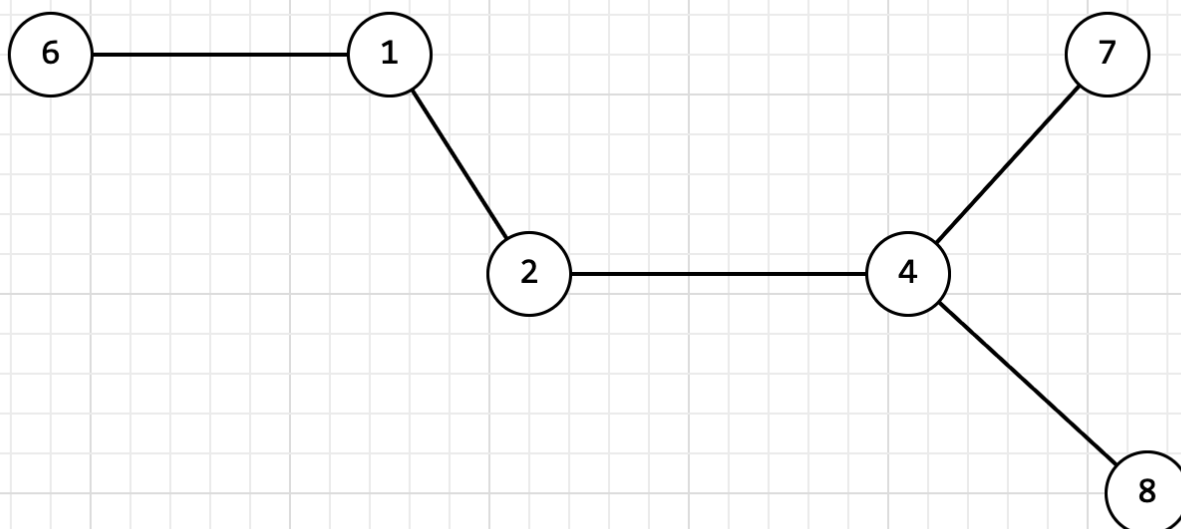
3. Если в дереве больше двух вершин, то п. 1, иначе выход.



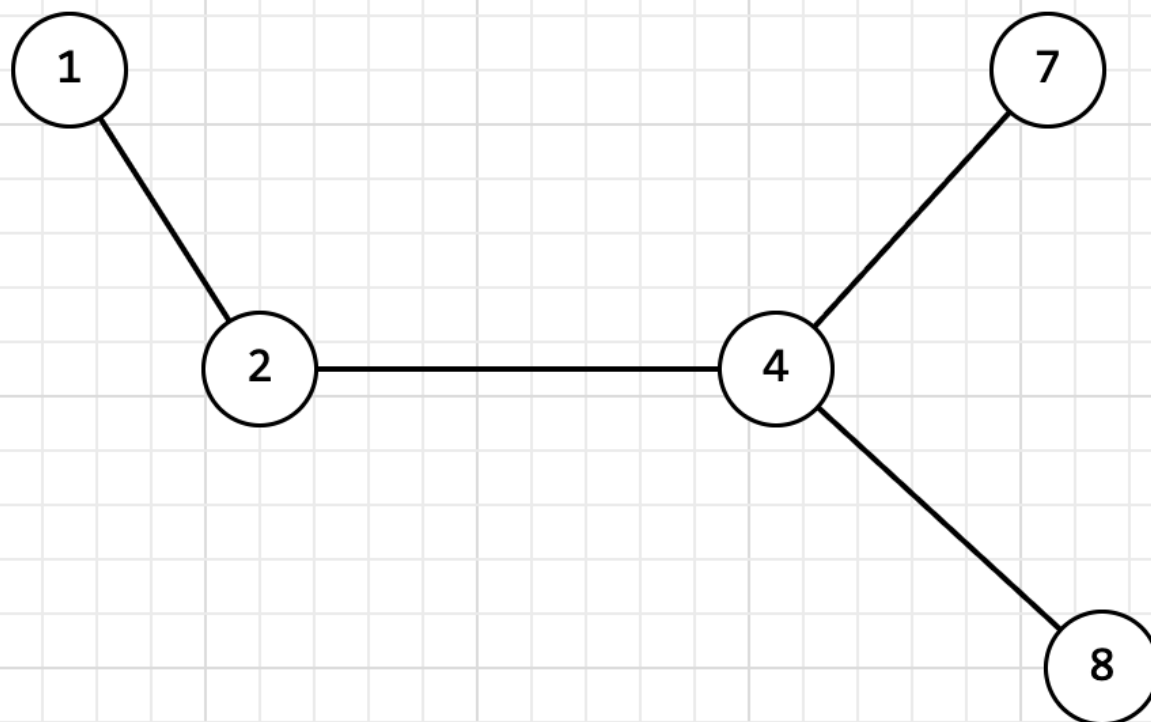
Лист с минимальным номером – 3. Смежная вершина – 2.



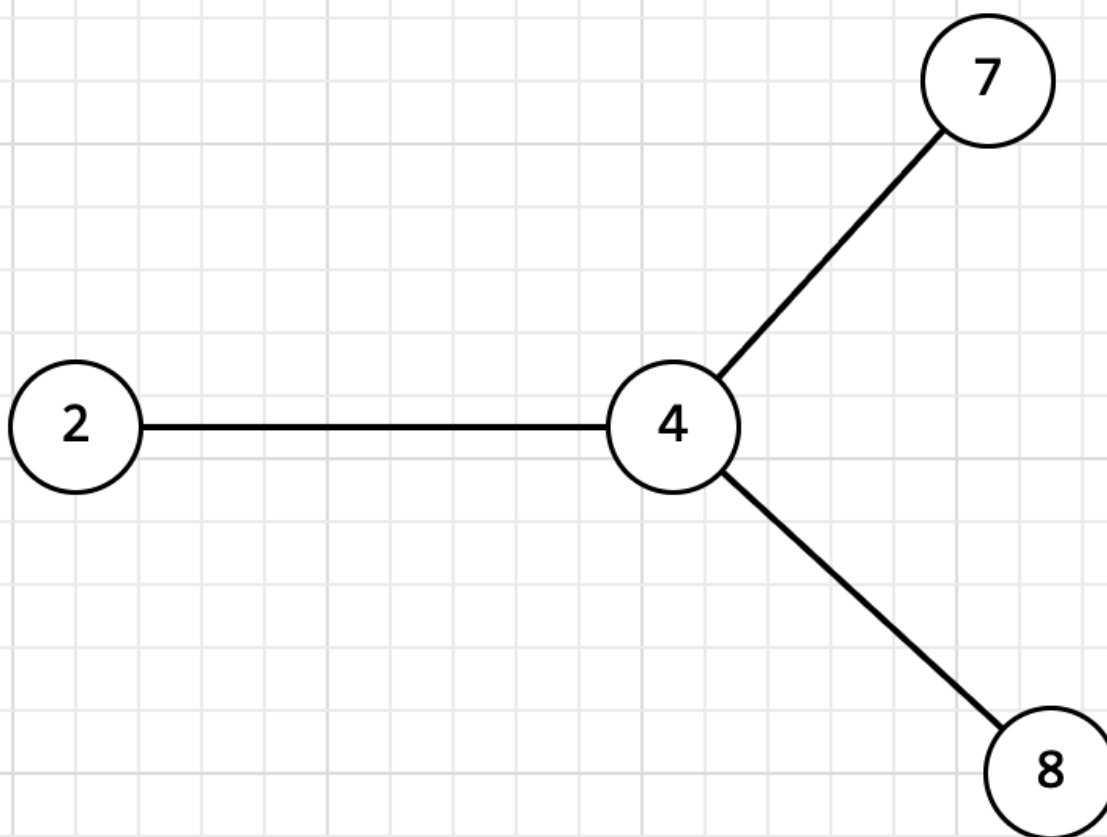
Лист с минимальным номером – 5. Смежная вершина – 4.



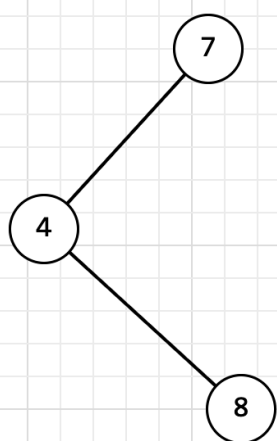
Лист с минимальным номером – 6. Смежная вершина – 1.



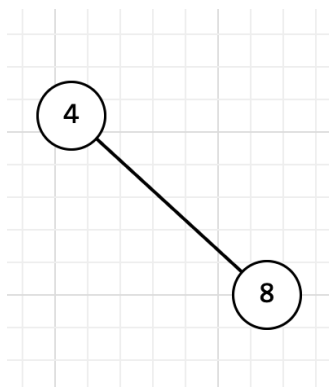
Лист с минимальным номером – 1. Смежная вершина – 2.



Лист с минимальным номером – 2. Смежная вершина – 4.



Лист с минимальным номером – 7. Смежная вершина – 4.



Ответ: (2, 4, 1, 2, 4, 4)

По любому коду Прюфера можно однозначно восстановить исходное дерево.

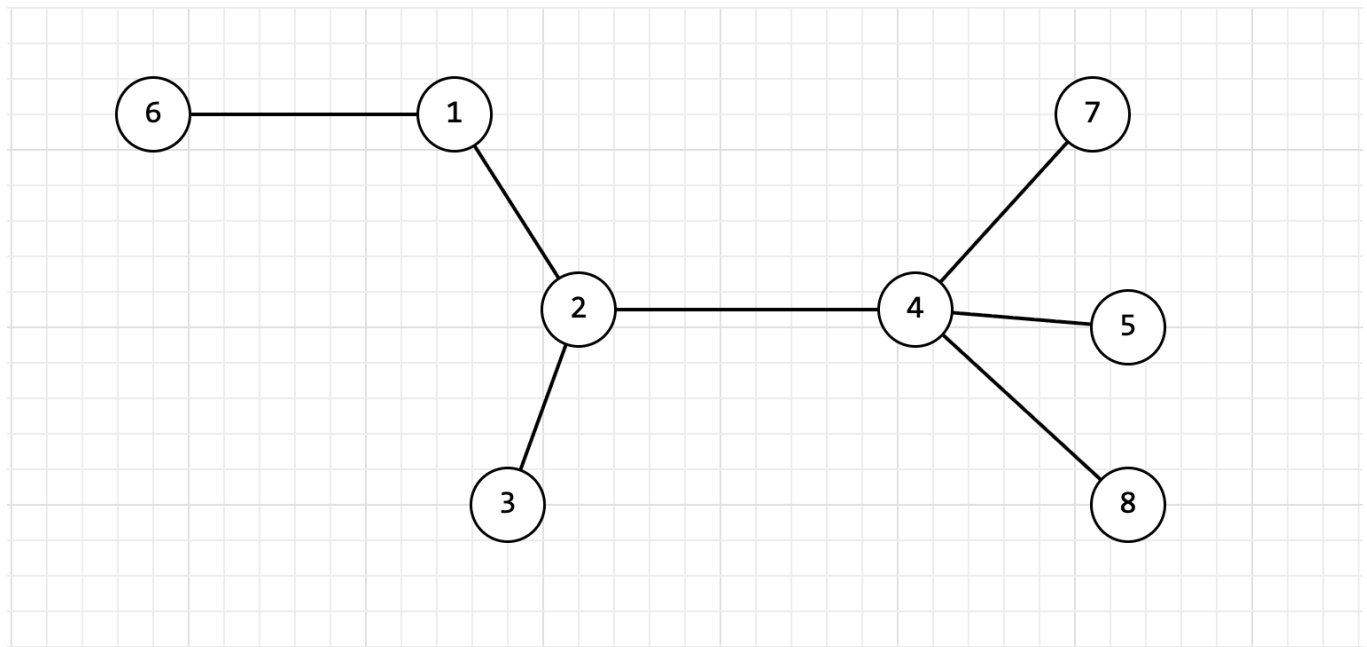
Заметим:

1. Листья исходного дерева в коде не встречаются.
2. Если вершина имеет степень 1, то такая вершина встречается в коде Прюфера ровно -1 раз.
3. Мы можем восстановить дерево, если на каждом шаге имели пару вершин - смежную и удаляемую. У нас есть последовательность смежных удаляемых вершин. Нужно восстановить по ней последовательность удаляемых.

(2, 4, 1, 2, 4, 4)

(3, 5, 6, 1, 2, 7)

Последнее ребро соединяет вершину с максимальным номером и единственную вершину с номером, отличным от максимального, которой нет во второй последовательности. В нашем случае это (8,4). Итак, нам нужно по верхней последовательности – коду Прюфера – восстановить нижнюю.



$() = (1, 2, \dots, n-3, n-2)$
 $(x_1, x_2, \dots, x_{n-3}, x_{n-2})$

Все числа во второй последовательности различны, так как каждый лист удаляем один

раз. На первом шаге мы брали лист с наименьшим номером. Лист не входит в последовательность Прюфера. Значит, нужно взять множество $X = 1, 2, \dots, n$, удалить из него все числа, которые входят в код Прюфера и из оставшихся выбрать минимальное число, это и будет x_1 . И мы имеем первое ребро $(x_1, 1)$. Далее удаляем 1 , из $P(T)$, x_1 из X и продолжаем действовать аналогично.

Восстановление дерева по коду Прюфера

1. Обозначим $() = [1, 2, \dots, n-3, n-2]$ — последовательность Прюфера, $X = (1, 2, \dots, n)$.
2. Выберем минимальное число из X , не содержащееся в $P(T)$.
3. Соединяем ребром вершину с номером и вершину, соответствующую первому числу из $P(T)$.
4. Удаляем из X , удаляем первое число из $P(T)$.
5. Если в X осталось два числа — соединяем ребром соответствующие вершины, иначе — п. 2.

Пример

$() = (2, 4, 1, 2, 4, 4)$
 $X = (1, 2, 3, 4, 5, 7,)$

1. Минимальное из X , которого нет в — это 3. Поэтому ребро $(3, 2)$. $() = (4, 1, 2, 4, 4)$,
 $X = (1, 2, 4, 5, 7,)$
2. Минимальное из X , которого нет в — это 5. Поэтому ребро $(5, 4)$. $() = (1, 2, 4, 4)$,
 $X = (1, 2, 4, 7,)$
3. Минимальное из X , которого нет в — это 6. Поэтому ребро $(6, 1)$. $() = (2, 4, 4)$,
 $X = (1, 2, 4, 7,)$
4. Минимальное из X , которого нет в — это 1. Поэтому ребро $(1, 2)$. $() = (4, 4)$,
 $X = (2, 4, 7,)$
5. Минимальное из X , которого нет в — это 2. Поэтому ребро $(2, 4)$. $() = (4)$, $X = (4, 7,)$
6. $(7, 4)$
7. Осталось $(4, 8)$

Полезная литература

1. [Курс лекций по Графам Знаний \(Knowledge Graphs\)](#)

Используются в машинном обучении:

- Дополненное глубинное обучение для анализа тональности высказываний.
- Предоставление новостей, рекомендованных конкретному пользователю.
- Машинный перевод.
- Для предоставления рекомендаций.

2. [Как графы знаний и LLM могут друг другу помочь](#)
3. Применение нейронных сетей для анализа графов со свойствами гомофилии и гетерофилии (<https://habr.com/ru/companies/fa/articles/828172/>)
4. В теории, использование графовой структуры может помочь ML-модели в выучивании закономерностей, однако на практике очень сложно внедрить графы в продакшен из-за ряда проблем. В статье рассказано о внедрении графов в реальную социальную сеть, с какими сложностями столкнулись и что усвоили.
(<https://arxiv.org/abs/2402.11139>)
5. [Курс "Машинное обучение на графах"](#)
6. Графовые нейронные сети (в частности, решение задач комбинаторной оптимизации), графовые базы данных, марковские сети, байесовские сети, диаграммы влияния и многое другое... (Вырожденностные графовые модели. Принципы и приложения – Луис Энрике Сукар)
7. Книга "Графовые нейронные сети на Python" – Лабонн М., Груздев А.В.
8. Книга "Теория графов" – О. Оре, "Теория графов" – Ф. Харари